

中国大学生计算机设计大赛



软件开发类作品文档简要要求

作品编号: _____

作品名称: _____ 心宇宙重塑: 房树人图像趣拼

作 者: _____ 曾帅洋 蹇佳成 梁杰

版本编号: _____ V 1.0

填写日期: _____ 2026/3/31

填写说明:

- 1、本文档适用于所有涉及软件应用与开发类的各个小类作品, 包括: (1) Web 应用与开发 (2) 管理信息系统 (3) 移动应用开发 (非游戏类) (4) 算法设计与应用 (5) 软件应用与开发专项赛;
- 2、本文档为简要文档, 不宜长篇大论, 需简明扼要, 建议设计二级目录, 逻辑性强;
- 3、一级标题采用二号黑体, 居中, 二级标题采用三号黑体, 靠左, 根据需要可以设计三级标题, 正文一律用五号宋体;
- 4、提交文档时, 以 PDF 格式提交本文档;

第一章 需求分析	5
第二章 概要设计	6
2.1 系统架构	6
2.2 适合的目标用户以及原因	12
2.3 主要性能	13
2.4 核心功能模块	13
2.4.1 页面管理模块	13
2.4.2 图片处理模块	13
2.4.3 拼图游戏模块	14
2.4.4 教程模块	16
2.4.5 音乐播放模块	16
2.4.6 心理报告生成模块	17
2.4.7 心理疗愈模块	18
2.4.8 管理员功能模块	18
2.4.9 数据存储模块	18
2.5 模块调用关系	19
2.6 人机交互界面	20
第三章 详细设计	26
3.2 数据存储设计	27
3.2.1 数据存储方案	28
3.2.2 数据结构设计	28
3.3 核心类设计	29
3.3.1 类图总览	29
3.4 关键算法设计	31
3.4.1 图像切割算法	31
3.4.2 碎片匹配算法	32
3.4.3 难度词条调度算法	33

3.5 用例与交互流程	34
3.5.1 拖拽操作流程	34
3.5.2 碎片放置流程	35
3.5.3 游戏完成检测流程	36
3.6 数据流程设计	38
3.6.1 系统数据流图	38
3.6.2 用户操作数据流	38
3.7 组件关系设计	39
3.7.1 组件依赖图	39
第四章 测试报告	40
4.1 测试环境	40
4.2 功能测试（黑盒测试）	41
4.2.2 兼容性测试	42
4.2.3 性能测试	44
4.2.4 功能测试结果汇总	44
4.3 白盒测试（代码级测试）	45
4.3.1 白盒测试详细设计	45
4.3.4 测试结论	47
4.3.2 测试用例设计	48
4.3.3 测试执行	50
4.4 问题修正记录	51
第五章 安装及使用	51
5.1 部署环境要求	51
5.1.1 服务器要求	51
5.1.2 浏览器要求	52
5.1.3 域名配置	52
5.1.4 环境变量配置	52
5.2 部署流程	53

5.2.1 前后端统一部署流程（腾讯云轻量级服务器）	53
5.3 使用说明	53
5.3.1 访问方式	53
5.3.2 典型使用流程	53
第六章 项目总结	55
6.1 开发历程与团队协作	55
6.1.1 用户调研	55
6.1.2 运营数据	57
6.2 克服的技术难点	58
6.2.1 移动端拖拽体验优化	58
6.2.2 前后端分离架构设计	58
6.2.3 用户行为数据分析	58
6.2.4 Canvas 性能优化	59
6.2.5 浏览器兼容性问题	59
6.2.6 AI 对话质量控制	59
6.3 能力提升与收获	59
6.4 后续升级规划	60
6.4.1 功能升级	60
6.4.2 技术升级	60
6.4.3 商业化探索	60
6.5 项目价值与意义	61

第一章 需求分析

在社会经济高速发展的当下，生活节奏加快、竞争压力增大，心理健康问题已成为全球性社会热点。根据世界卫生组织（WHO）2025 年 9 月发布的最新数据，全球超过 10 亿人患有精神障碍，每 7 人中就有 1 人受其困扰，其中焦虑症和抑郁症最为常见，仅 2021 年全球就有 3.59 亿焦虑症患者和 2.8 亿抑郁症患者，且这类疾病已成为导致长期残疾的第二大原因，给个人和社会带来巨大负担。

聚焦我国，心理健康问题同样形势严峻。《2023 年度中国精神心理健康》蓝皮书数据显示，中国成人抑郁风险检出率达 10.6%；国际权威期刊《英国精神病学杂志》2025 年 1 月发布的研究表明，1990-2021 年间，我国抑郁症患病人数增幅达 54%，从 3440 万跃升至 5310 万，焦虑症患病人数增幅为 31.2%，其中 10-19 岁青少年焦虑患病率达 4.5%，为各年龄组最高，55 岁以上老年人抑郁率达 6.5%，是总人群的 1.7 倍。此外，最新调查显示我国成年人各类精神疾病（不包含痴呆）的终生患病率为 16.6%，由精神障碍造成的疾病负担占有所有非传染性疾病负担的 13%，但接受正规治疗的患者比例不足 10%。

但传统心理健康服务面临诸多瓶颈：专业心理咨询资源稀缺且服务门槛高，普通大众难以获得便捷、低成本的心理支持；经典的房树人测验（HTP）作为投射性心理评估工具，虽能通过房屋、树木、人物三元素的绘画分析，揭示个体潜意识中的自我概念、情感状态与人际关系模式，且已有研究证实其 50 项绘画特征中 39 项可显著预测精神障碍，具备科学的评估基础，但该测验依赖专业分析师解读，存在评估标准主观、解读周期长等问题，无法满足大众日常化、轻量化的心理调节需求。

与此同时，游戏化疗愈已成为心理健康服务的重要发展方向。北京师范大学相关研究指出，游戏作为直接作用于人类心理、情绪与认知的媒介形式，在缓解焦虑、改善注意力缺失等方面已展现出明确且显著的干预效果，全球已有 10 款数字疗法游戏获批用于治疗焦虑、抑郁等疾病。拼图游戏作为其中的典型代表，其疗愈价值已得到实证支撑——Mind Lab International 2017 年的研究显示，拼图可使压力水平降低 28%，印度一项 2026 年 3 月发布的随机临床试验也证实，拼图游戏能有效降低儿童在医疗操作中的焦虑和疼痛感。但现有拼图类产品多以娱乐为核心，缺乏心理学理论支撑，仅能满足休闲需求，无法实现心理减压与自我觉察的深层价值。

在此背景下，将房树人测验的心理学内核与拼图游戏的互动性、娱乐性相结合，开发一

款兼具趣味性与心理疗愈功能的软件，成为解决大众日常心理健康需求的有效途径。

现经调研显示,目前市面上存在的传统网页拼图游戏存在题材单一、缺乏心理学价值、难度固定、移动端体验差、隐私保护不足等问题,分析结果如表 1 所示。

表 1 竞品分析

对比维度	竞品（传统网页拼图）	本作品
题材特色	网络图片、游戏图片	房树人心理投射主题图
心理学价值	无心理学理论支撑	深度融合 HTP 测验理论
难度灵活性	固定难度或少量选项	多级难度+三种增强词条
图片来源	固定内置图片	内置主题图+自定义上传
隐私保护	需上传服务器	纯前端处理，本地运行
移动端体验	PC 移植，操作不便	原生触摸交互设计
使用门槛	需搜索、可能注册	打开网址即用
心理引导	无	内置问卷，引导自我觉察

第二章 概要设计

2.1 系统架构

本系统采用前后端分离的 B/S（浏览器/服务器）架构。前端采用纯 HTML5+CSS3+JavaScript 实现，后端基于 Node.js+Express 框架，数据层使用 SQLite 数据库。整体采用三层架构设计，包含表现层（UI 界面）、业务逻辑层（游戏核心逻辑+后端 API）、数据层（SQLite 数据库+本地存储），架构如图 1 所示。

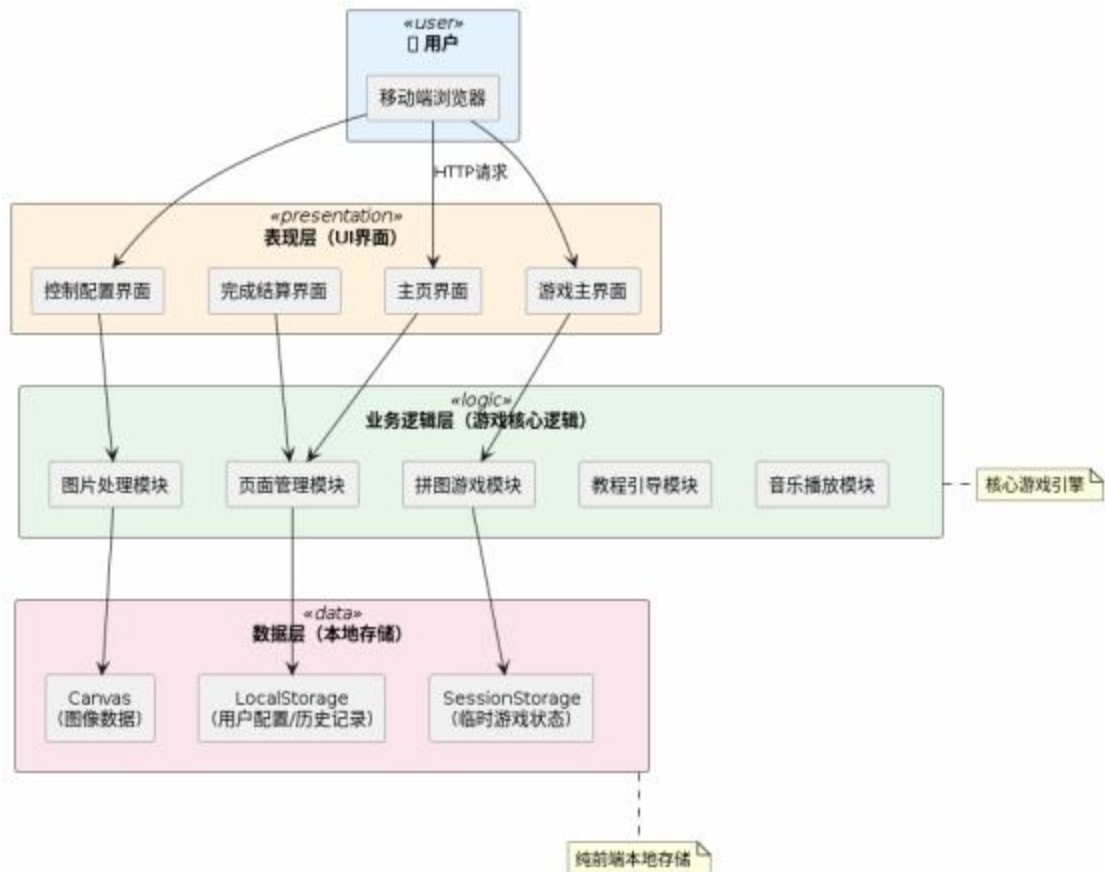


图 1 系统架构图

前端层： - 用户界面：HTML5 Canvas 绘图、响应式布局、触摸交互 - 客户端逻辑：拼图游戏控制、图片处理、音乐播放 - 本地存储：LocalStorage 保存用户配置和游戏进度

后端层： - Web 服务器：Express 框架，端口 3001 - 核心模块： - 拼图引擎（puzzle_engine.js）：服务端状态管理、游戏逻辑验证 - 图片校验（image_validator.js）：房树人三要素检测 - 数据分析（analytics_store.js）：用户行为记录与分析 - 管理员功能（admin-healing.js）：数据导出、权限管理 - API 接口：RESTful API 设计，支持 CORS 跨域

数据层： - SQLite 数据库（behavior_analytics.db）： - users 表：用户信息 - game_sessions 表：游戏会话记录 - action_logs 表：用户操作日志 - report_logs 表：心理报告记录 - healing_sessions 表：疗愈会话记录 - healing_messages 表：疗愈对话记录

外部服务： - DeepSeek AI API：心理报告生成、疗愈对话 - 腾讯云轻量级服务器：前后端统一部署 - 腾讯云服务器：后端 API 服务部署

系统主要涵盖 9 个核心功能模块，包括页面管理模块、图片处理模块、拼图游戏模块、教程引导模块、音乐播放模块、数据存储模块、心理报告生成模块、心理疗愈模块、管理员功能模块，各模块功能如下：

- 1.图片上传处理：支持用户上传自定义图片或选择内置房树人主题图，支持图片三要素（房、树、人）智能校验。
- 2.多级难度设置：提供多种难度等级（ 2×2 至 6×6 ）及增强选项（旋转、隐藏、捣蛋鬼）。
- 3.智能拼图游戏：触摸拖拽交互、实时反馈、完成检测、服务端状态管理、撤销功能（最多 300 步）。
- 4.辅助功能：参考图查看、自动求解、重新打乱、碎片旋转。
- 5.音乐播放：本地音乐播放器，支持背景音乐。
- 6.结果保存：完成时间、步数统计、行为数据记录。
- 7.心理报告生成：基于 AI（DeepSeek）生成个性化心理分析报告，结合用户近期行为数据提供更精准的分析。
- 8.心理疗愈对话：基于心理报告的 AI 疗愈对话系统，支持 3 轮问答，提供情绪安抚和心理引导。
- 9.管理员功能：管理员登录、数据导出（Excel 格式）、用户行为数据统计分析。

功能结构图如图 2 所示。

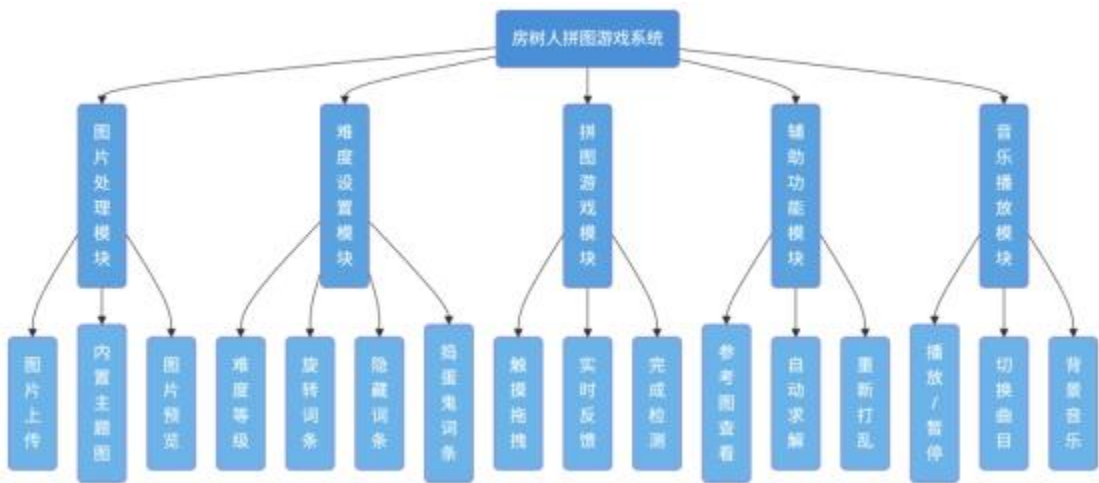


图 2 功能结构图

游戏设 2×2 至 6×6 六个难度等级，并提供拼图旋转、隐藏、捣蛋鬼等增强选项：简单模式让初学者在轻松中缓解焦虑；困难模式则锻炼抗挫能力与专注力，满足不同心理需求。

在房树人心理学相关内容中，房树人测验通过分析绘画者对房子、树木、人物的描绘方式，揭示其潜意识中的心理状态与人格特征，相关心理学分析示例如图 3、图 4 所示。

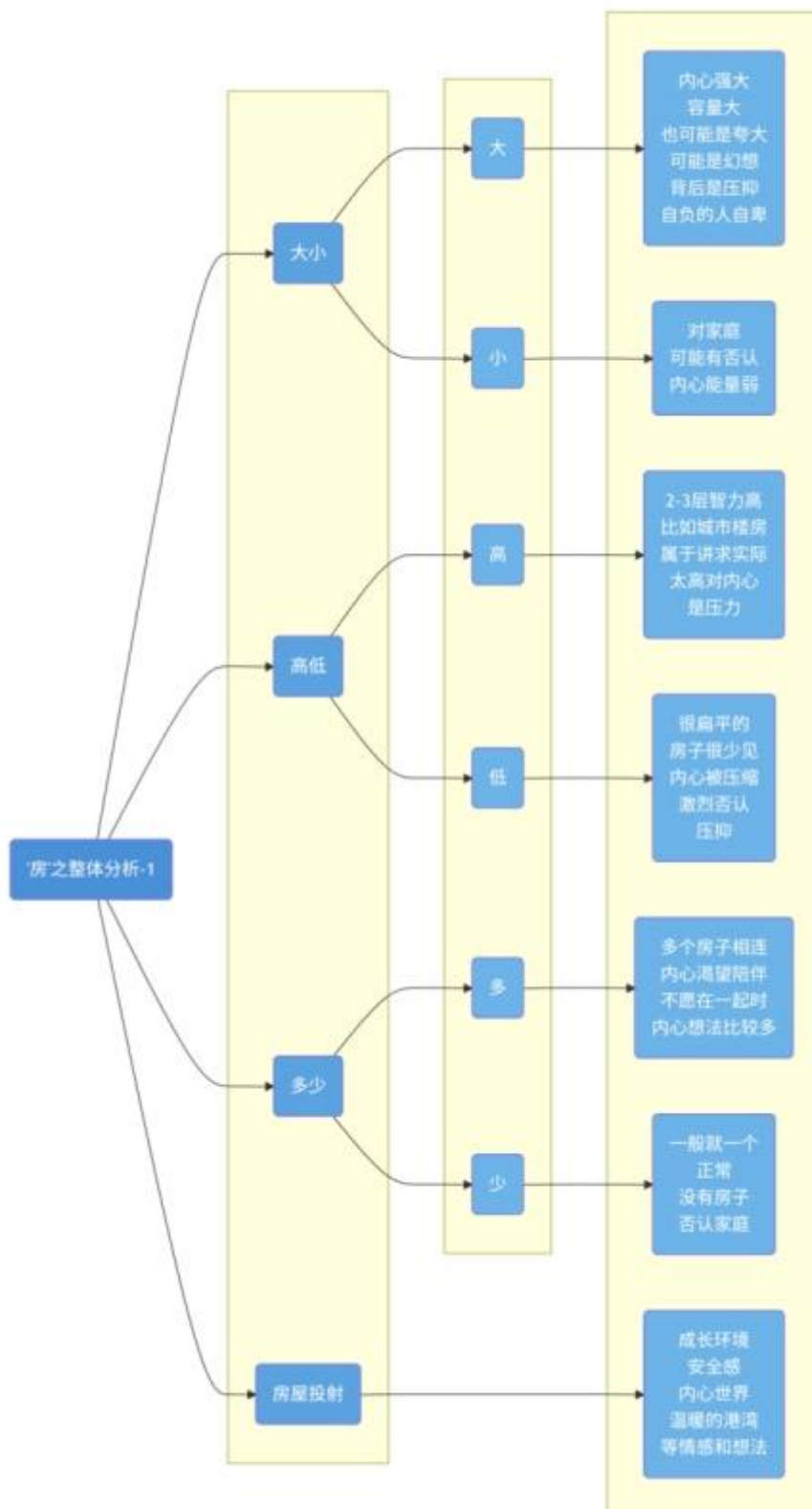


图 3 房树人关于房屋心理学分析 1

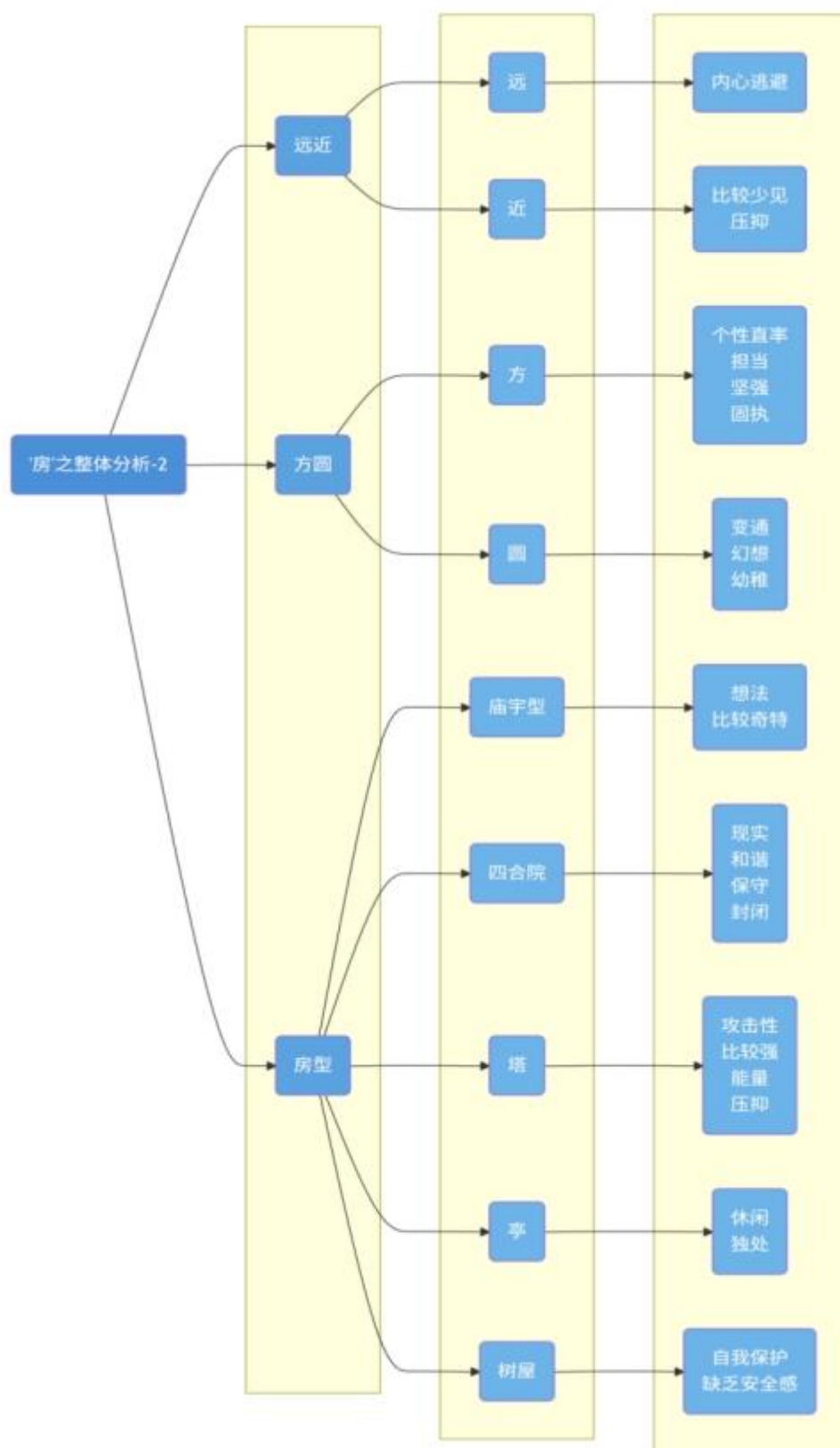


图 4 房树人关于房屋心理学分析 2

从心理学角度有房子、树、人 3 大内涵，具体内容如下：

（1）房子

房子是个体出生成长的地方，也反映出个体对家庭、家族关系的看法。通过对屋顶、窗户、门、地面线等构成部分的分析，可以了解到个体在家庭中自我形象，空想与现实的关系，家庭亲子关系，安全感，家庭与环境的关系等。

（2）树

树表现的是个体自己几乎无意识感到的自我形象、姿态，表示其内心的平衡状态，由此可显示出个体的精神及性的成熟性；当然树还具有的直接含义是个体与环境的关系，具有生命意义的象征，所以可称为生命树，表现出个体生命成长的历程。

（3）人

最明显反映人的形象。所以画人时，会自动用心理防御机制。人反映的是自我现实像。心理上的和躯体上的自我，当然还有表现个体的理想像，印证自我的人格内容。不画人：表示对人的彻底否定，有自杀倾向。

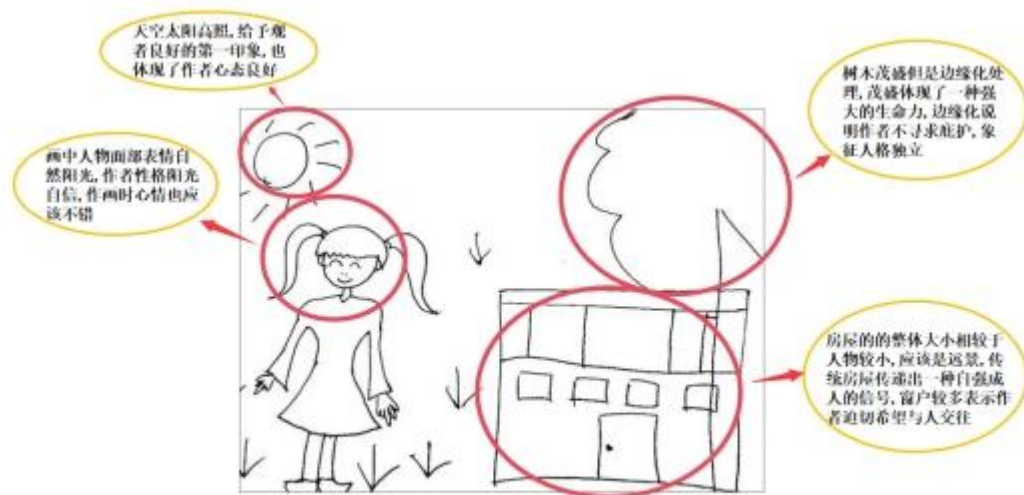


图 5 房树人心理学绘画分析

2.2 适合的目标用户以及原因

（1）青少年群体(12-18 岁)：缓解焦虑及非语言表达。

（2）青年群体(18-35 岁)：碎片时间探索减压。

- (3) 中年群体 35-55 岁：工作减压，促进亲子互动。
- (4) 中老年群体 55 岁以上：延缓认知老化，低难度友好。
- (5) 学校心理中心：降低学生防御，辅助心理教育。
- (6) 心理咨询机构：数字化评估，缓解紧张情绪。

2.3 主要性能

响应速度：前后端分离架构，前端图片处理延迟<100ms，后端 API 响应<100ms

兼容性：支持主流移动端浏览器（Chrome、Safari、Edge）

流畅度：流畅度：触摸交互响应迅速，无卡顿

2.4 核心功能模块

2.4.1 页面管理模块

页面管理模块负责管理系统的多个核心界面，实现流畅的页面切换和状态管理。

功能： - 管理主页、配置页、游戏页、完成页四个主要页面的切换与显示 - 管理管理员登录、管理员面板、心理疗愈等模态框的显示与隐藏 - 控制报告列表、疗愈对话等交互界面的状态

实现： - 通过 CSS 类控制页面显示/隐藏 - 使用模态框（Modal）实现弹窗交互 - 支持响应式布局，适配移动端和桌面端

核心接口： - showPage(pageId) - 显示指定页面 - showModal(modalId) - 显示模态框 - hideModal(modalId) - 隐藏模态框

2.4.2 图片处理模块

功能：图片选择、上传、预览、切割、三要素校验

核心技术：File API 读取、Canvas 绘制与切割、AI 图像识别

数据流向：用户上传 → File 读取 → 三要素校验 → Canvas 绘制 → 切割为碎片

前端处理： - 支持本地图片上传（File API） - 内置 4 张房树人主题图片 - Canvas 图片缩放和切割 - 碎片 Base64 编码存储

后端校验（image_validator.js）： - 房树人三要素检测（房屋、树木、人物） - 基于

模块内容如图 6 所示。

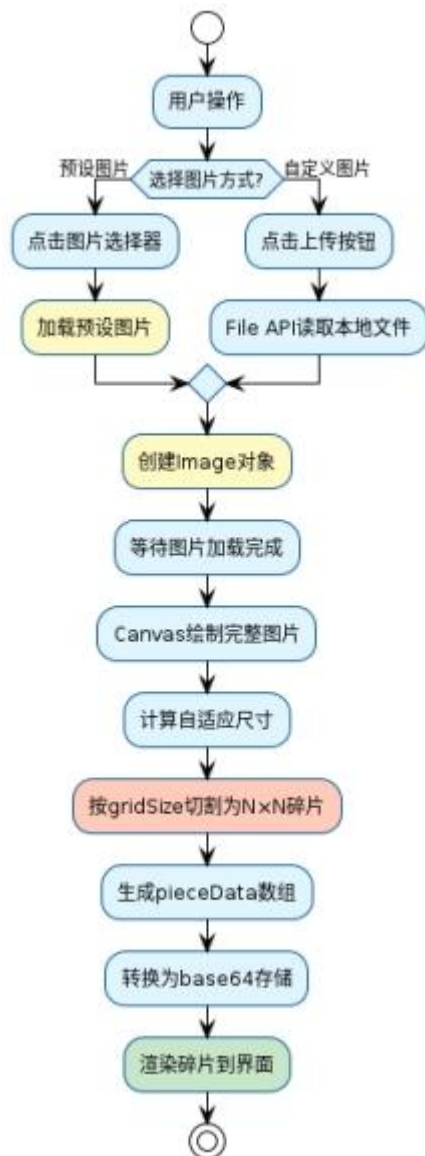


图 6 图片处理模块流程图

2.4.3 拼图游戏模块

拼图游戏模块是系统的核心控制模块,采用前后端分离架构,负责游戏的整体流程管理、碎片交互处理和完成状态检测。

前端部分 (puzzle-client.js) : - 用户交互处理: 拖拽、点击、触摸事件 - 游戏界面渲染: Canvas 绘制、动画效果 - 状态同步: 与后端 API 通信, 获取游戏状态

后端部分 (puzzle_engine.js) :

核心属性： - 原始图片对象 - 难度等级(2-6) - 碎片数据数组(board、tray、hiddenPool)
- 难度词条（旋转/隐藏/捣蛋鬼） - 游戏状态（playing/completed） - 撤销历史（最多 300 步）

核心方法：

- (1) createGame() - 创建游戏。生成 2×2 至 6×6 的拼图，初始化碎片位置。
- (2) executeAction() - 执行动作。处理用户操作（放置、移动、旋转等）。
- (3) place_from_tray() - 从托盘放置碎片到游戏板。
- (4) move_cell() - 移动格子中的碎片。
- (5) rotate_piece() - 旋转碎片（需启用旋转词条）。
- (6) undo() - 撤销上一步操作。
- (7) solve() - 自动完成拼图。
- (8) checkCompletion() - 完成检测。判断所有碎片是否正确放置。
- (9) trigger_trickster() - 捣蛋鬼调度。随机移动已放置的碎片（15%触发概率）。
- (10) revealAdjacentHidden() - 隐藏碎片显示机制。放置碎片后自动显示相邻隐藏碎片。

数据库集成： - 每个游戏会话记录到 game_sessions 表 - 每个用户操作记录到 action_logs 表 - 自动统计行为指标（碎片放置顺序、操作时间间隔、修改次数）

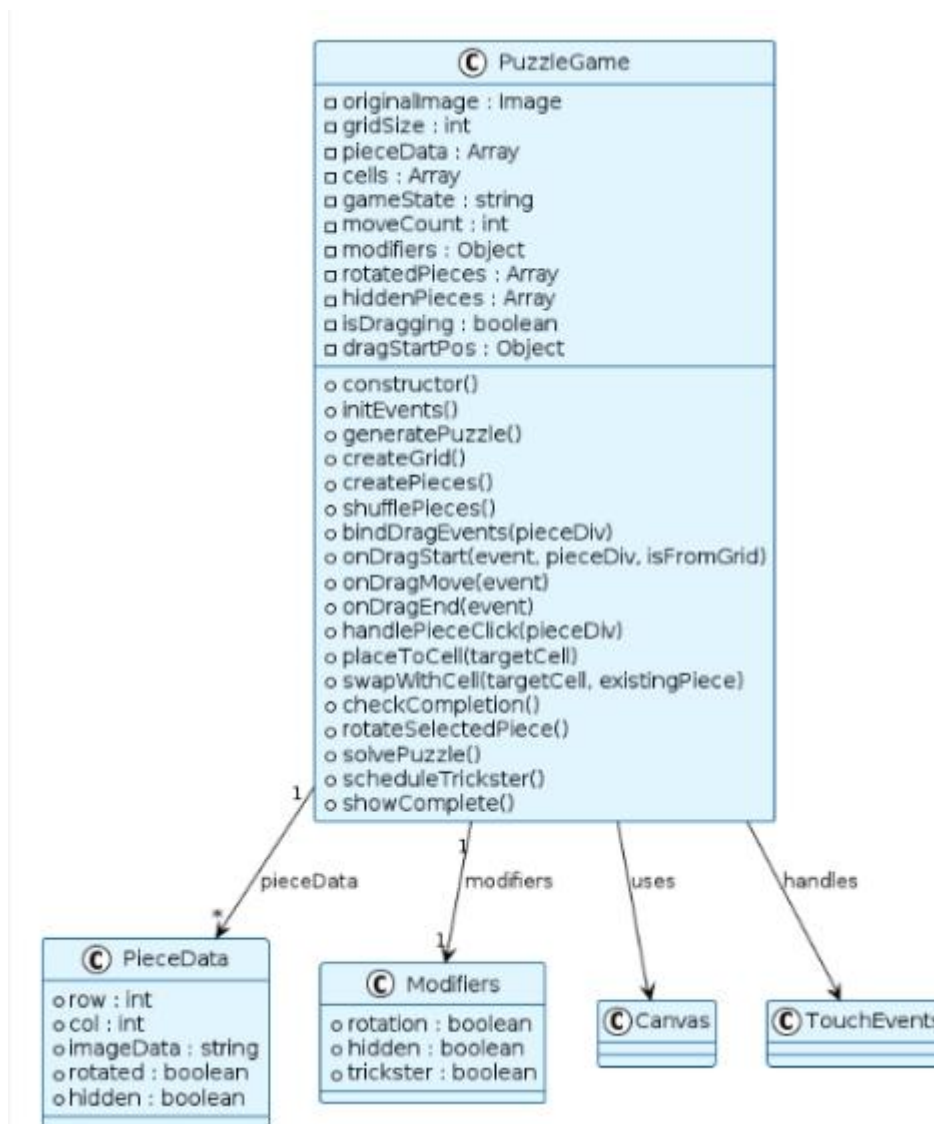


图 7 拼图游戏模块类

2.4.4 教程模块

功能：新手引导与操作提示

实现：遮罩层+高亮+提示文本

2.4.5 音乐播放模块

功能：本地音乐播放与控制

核心方法：

- (1) 播放/暂停
- (2) 切换曲目

(3) 快进/快退

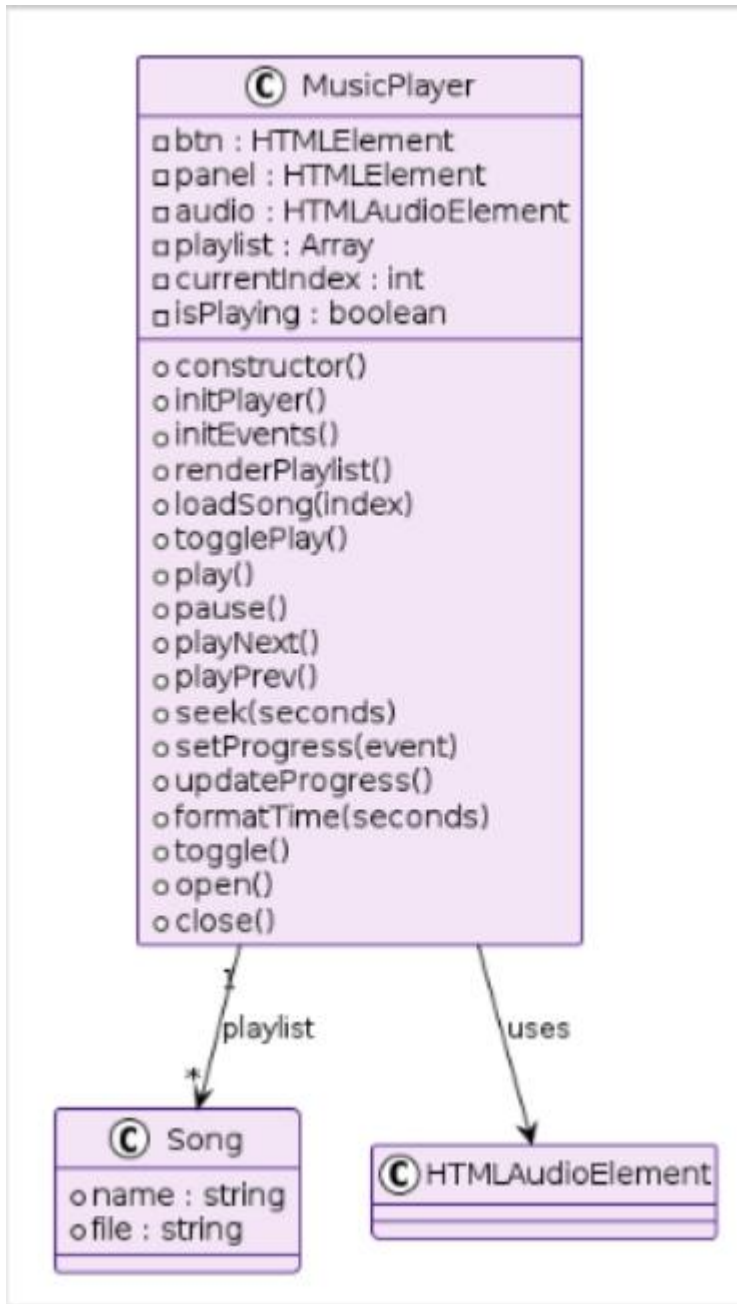


图 8 音乐播放模块类图

2.4.6 心理报告生成模块

功能：基于 AI 生成个性化心理分析报告

核心技术： - DeepSeek AI API 集成 - 用户行为数据分析 - 近期游戏记录统计（最多 12 局，14 天内）

数据流程： 1. 收集本次游戏数据（用时、步数、修改次数等） 2. 查询用户近期行为

摘要（平均用时、平均步数、高犹豫动作占比等） 3. 构建 AI 提示词（包含本次数据+近期摘要） 4. 调用 DeepSeek API 生成报告 5. 保存报告到数据库（report_logs 表）

报告内容： - 心理状态分析 - 行为模式解读 - 个性化建议 - 历史对比（如有近期数据）

2.4.7 心理疗愈模块

功能：基于心理报告的 AI 对话疗愈系统

核心特性： - 3 轮问答限制 - 基于报告内容的个性化回复 - 情绪安抚和心理引导 - 会话状态管理

对话流程： 1. 用户选择已生成的心理报告 2. 系统创建疗愈会话（healing_sessions 表） 3. 第一轮：AI 阅读报告，回答用户提问，安抚情绪 4. 第二轮：继续安抚和引导 5. 第三轮：总结对话，给出结论 6. 提交用户信息（可选匿名）

数据记录： - 所有对话保存到 healing_messages 表 - 用户信息保存到 healing_sessions 表 - 支持管理员导出疗愈数据

2.4.8 管理员功能模块

功能：数据管理和统计分析

核心功能： 1. 管理员登录验证（默认密码：123456） 2. 数据导出（Excel 格式） - 游戏会话数据 - 心理报告数据 - 疗愈对话数据 3. 数据可视化标记 - 自动标红”有问题”和”有大问题”记录 - 按时间排序 - 包含完整用户信息

技术实现： - 密码验证（可通过环境变量配置） - ExcelJS 库生成 Excel 文件 - 数据库查询和聚合 - 前端模态框交互

2.4.9 数据存储模块

功能：用户行为记录和数据持久化

数据库表结构： 1. users 表：用户唯一标识、首次/最近访问时间 2. game_sessions 表：游戏会话完整记录 3. action_logs 表：用户操作详细日志 4. report_logs 表：心理报告记录 5. healing_sessions 表：疗愈会话记录 6. healing_messages 表：疗愈对话记录

核心功能： - 自动记录用户行为 - 生成近期行为摘要 - 支持数据查询和统计 - 隐私

保护（匿名 client_id）

2.5 模块调用关系

系统采用前后端分离的分层调用架构。前端 **PuzzleGame** 核心控制模块统管客户端业务逻辑，协调各功能模块协同工作；后端 **PuzzleEngine** 提供服务端状态管理和游戏逻辑验证。

前端模块： - **ImageProcessor** 图片处理层负责图像加载、切割等预处理操作 -

TouchHandler 交互处理层捕获用户触摸事件并转化为游戏操作指令 - **StorageManager** 数据存储层提供本地持久化服务 - **MusicPlayer** 和 **Tutorial** 辅助功能层分别管理音频播放和新手引导 - **AdminManager** 和 **HealingManager** 管理管理员功能和心理疗愈功能 - 页面管理层控制界面切换与状态同步

后端模块： - **PuzzleEngine** 拼图引擎提供服务端游戏状态管理 - **AnalyticsStore** 数据分析模块记录用户行为并生成统计摘要 - **ImageValidator** 图片校验模块验证房树人三要素 - **API** 路由层处理前端请求并返回响应

调用时序： 整体调用时序遵循“用户操作→前端事件捕获→API 请求→后端处理→数据库记录→状态更新→API 响应→前端界面反馈”的闭环机制。通过清晰的模块依赖关系、RESTful API 接口和标准化数据格式，确保系统响应的实时性、稳定性和可维护性，为流畅的拼图游戏体验和完整的心理分析功能提供可靠的技术保障。

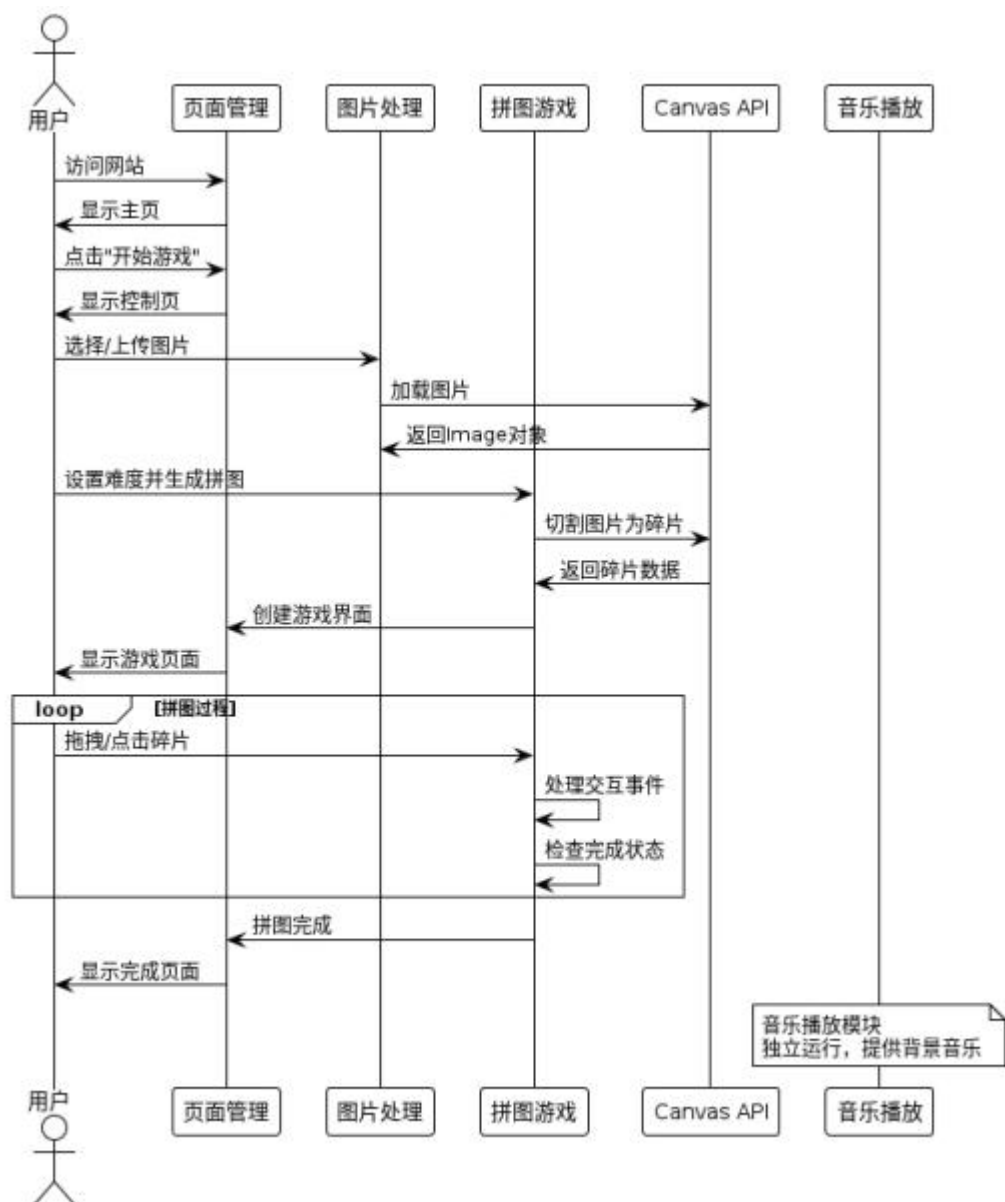


图 9 模块调用关系时序图

2.6 人机交互界面

本系统采用移动端优先的响应式设计，主要包含 4 个核心界面和多个功能模态框，形成完整的用户交互闭环。各界面具体内容如下所示。

(1) 主页。包含游戏规则介绍、房树人心理学简介、开始按钮以及管理员入口（右上角 按钮）。如图 10 所示。



图 10 主菜单界面

(2) 控制页：图片选择器（内置主题图/自定义上传）+ 难度设置（2×2 至 6×6）+ 难度词条（旋转/隐藏/捣蛋鬼）+ 生成拼图按钮。如图 11 所示



图 11 配置页(拼图控制)

(3) 游戏页：拼图网格 + 碎片容器 + 操作按钮（打乱/翻转/求解/撤销）+ 参考图按钮 + 音乐播放控制。如图 12 所示

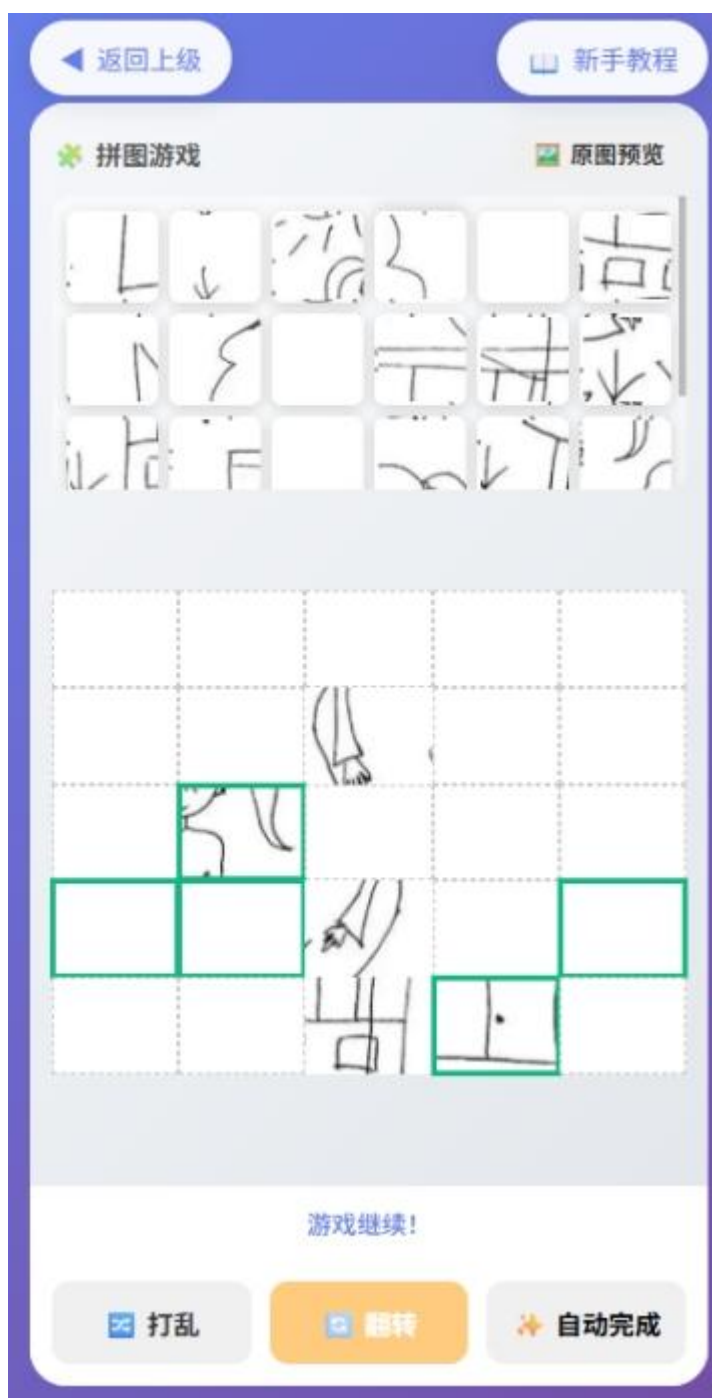


图 12 游戏界面功能

(4) 完成页：统计信息（用时/步数） + 再玩/返回按钮 + 生成心理报告按钮 + 查看心理报告按钮（带未读徽章） + 心理疗愈按钮。如图 13 所示



图 13 结算界面

(5) 功能模态框： - 管理员登录模态框：密码验证 - 管理员面板模态框：数据导出功能 - 心理报告列表模态框：显示用户所有报告 - 心理疗愈列表模态框：选择报告开始疗愈 - 心理疗愈对话模态框：AI 对话界面，显示问题计数 - 用户信息提交模态框：疗愈结束

后提交信息（可选匿名）

交互特点(如图 14 所示):

全触摸优化：支持拖拽、点击双模式

实时反馈：碎片正确/错误状态即时显示，按钮加载状态提示

流畅动画：CSS 过渡效果，提升体验

未读通知：报告未读徽章提醒

状态管理：会话状态颜色编码（绿色开始/黄色继续/白色查看）

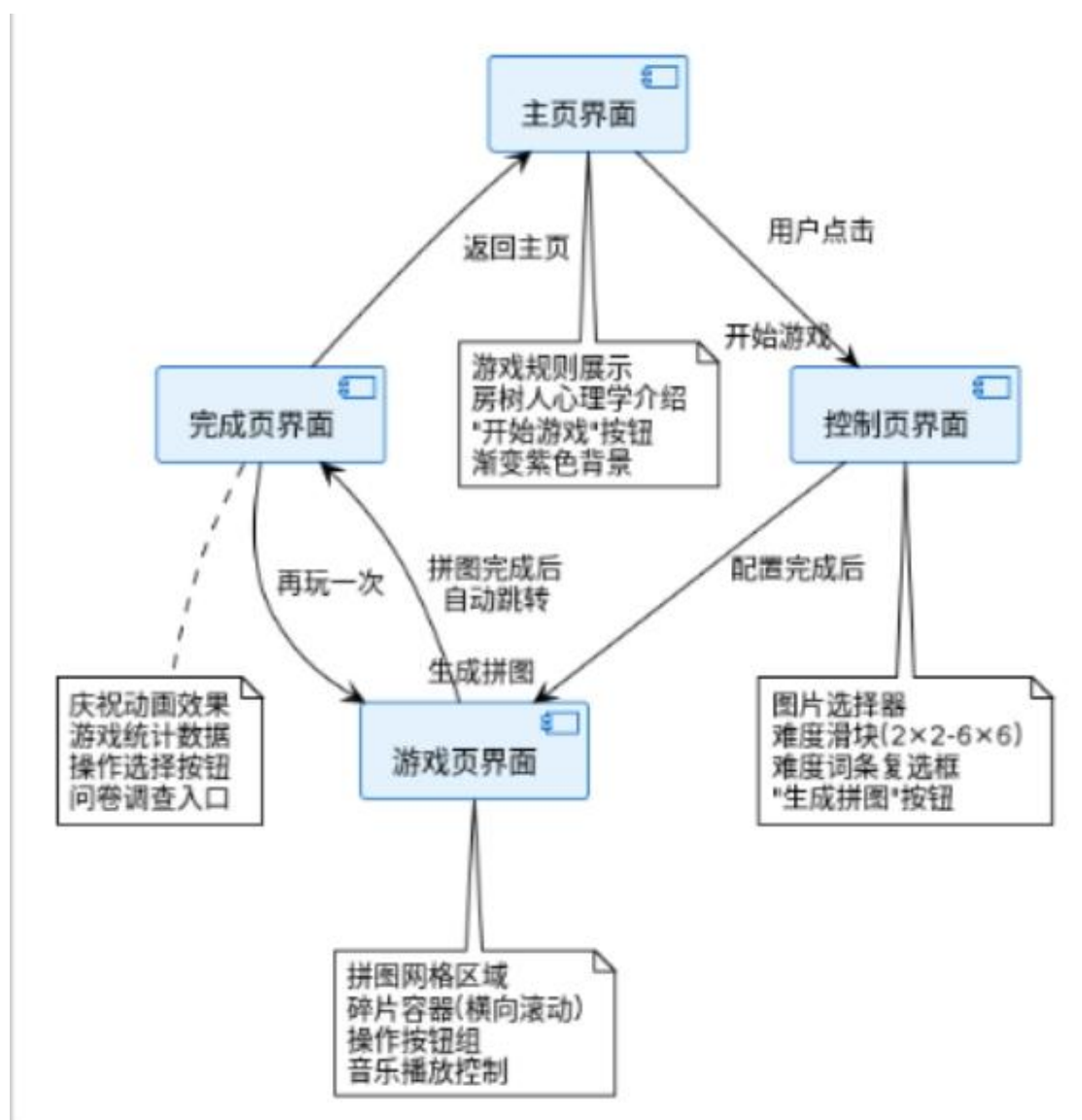
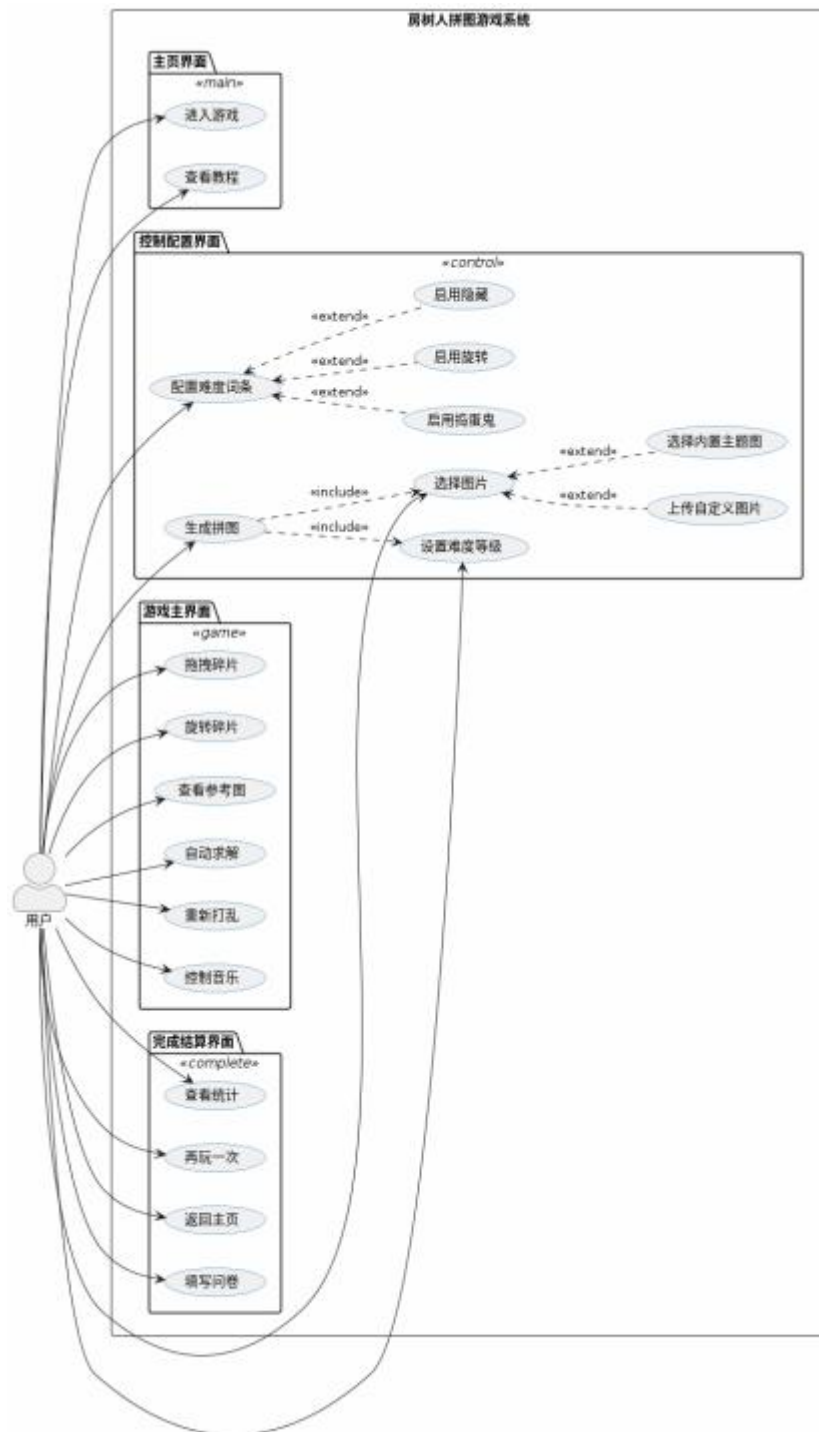


图 14 人机交互界面组件图

第三章 详细设计



3.1 界面设计

图 15 界面设计用例图

系统包含四个核心界面，形成完整的用户交互闭环(如图 16 所示)：

- (1) 主页界面：用户进入系统的第一个接触点，展示游戏规则介绍、房树人心理学简

介，以及” 开始游戏” 入口按钮。

(2) 控制配置界面：提供图片选择器（内置主题图/自定义上传）、难度滑块（2×2 至 6×6）、难度词条复选框（旋转/隐藏/捣蛋鬼）及” 生成拼图” 按钮。

(3) 游戏主界面：采用上下分区布局，上方为拼图网格区域，下方为碎片容器（横向滚动）， 底部设置操作按钮组（参考图/打乱/求解）及音乐播放控制。

(4) 完成结算界面：展示庆祝动画与统计数据（用时/步数），提供三个操作选择（再玩一次/更换图片/返回主页）及问卷调查入口。

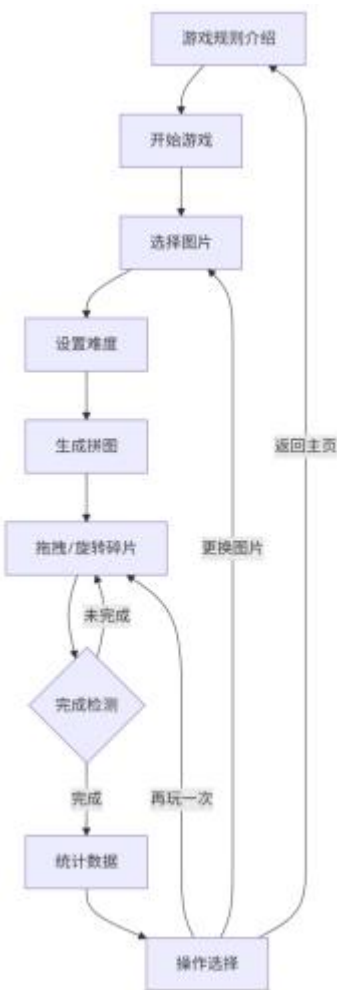


图 16 游戏界面流程图

3.2 数据存储设计

系统采用前后端分离的数据存储方案，充分利用浏览器本地存储技术，确保用户隐私安全的同时提供流畅的游戏体验。通过合理的数据结构设计，实现游戏状态的持久化保存和快速恢复。

3.2.1 数据存储方案

浏览器本地存储技术: LocalStorage 存储用户配置和历史记录, SessionStorage 存储临时游戏状态,如图 16 所示。

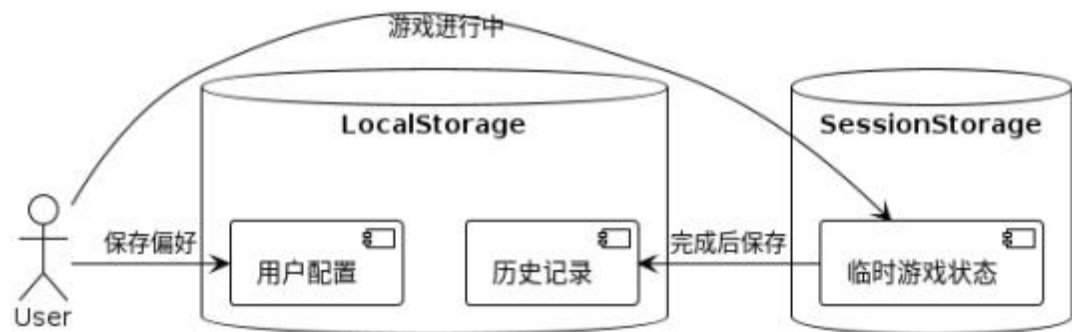


图 17 存储方案用例图

3.2.2 数据结构设计

系统采用四种核心数据结构来管理游戏的各项数据,如图 18 所示:

(1) 游戏配置数据: 存储用户选择的图片信息、自定义上传的图片数据、难度等级设置、已启用的难度词条选项以及音乐播放开关状态。

(2) 游戏状态数据: 记录当前游戏的运行状态、游戏开始时间、移动步数统计、所有碎片的位置和属性信息、网格布局数据以及各难度词条的实时状态。

(3) 游戏历史记录: 保存每局游戏的唯一标识、完成所用时间、总移动步数、游戏难度等级以及游戏完成的时间戳。

(4) 用户偏好设置: 包含界面主题风格、音效音量大小、背景音乐音量以及是否显示新手教程引导。



图 18 数据结构类图

3.3 核心类设计

3.3.1 类图总览

本系统采用面向对象的设计思想，共设计了 6 个核心类，分别承担不同的功能职责。整体架构遵循单一职责原则和依赖倒置原则，形成清晰的分层结构。核心控制层由 **PuzzleGame** 类统管全局；数据处理层包含 **ImageProcessor** 图像处理类；交互处理层由 **TouchHandler** 类

负责；辅助功能层包含 **MediaPlayer** 音乐播放类、**Tutorial** 教程引导类；基础设施层由 **StorageManager** 数据存储类提供支撑。各类间通过标准化接口进行交互，确保系统的可扩展性和可维护性。类图总览如图 20 所示：

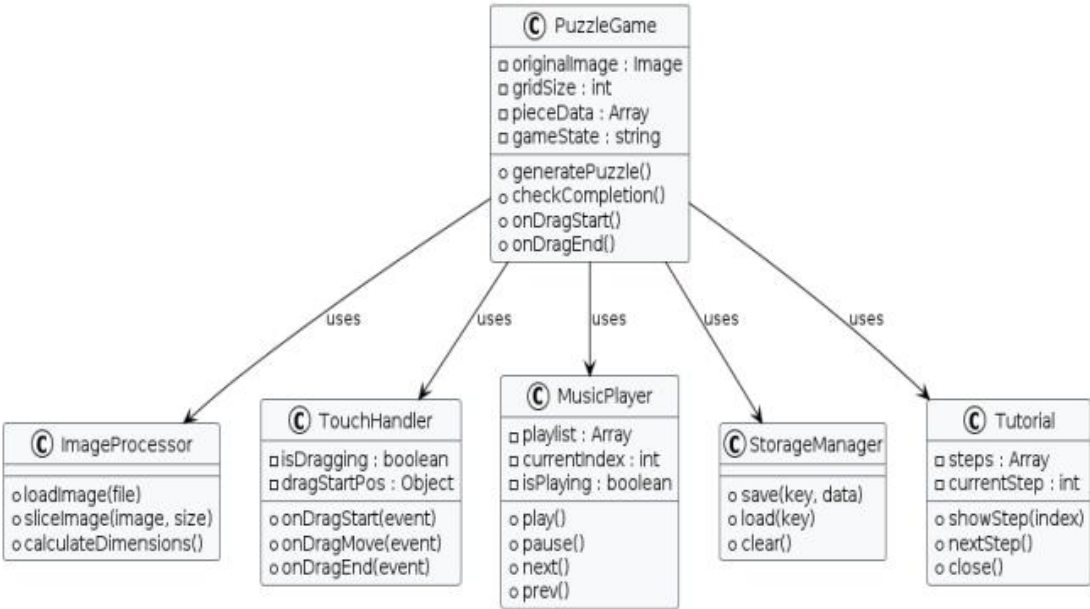


图 20 总览类图

类与对应功能如表 2 所示。

表 2 映射表

类名	功能描述	关键方法	备注
PuzzleGame	拼图游戏主控制类	generatePuzzle(), checkCompletion()	核心类，管理游戏状态和流程
ImageProcessor	图像处理类	loadImage(), sliceImage()	负责图片加载、缩放和切割
TouchHandler	触摸交互处理类	onDragStart(), onDragMove(), onDragEnd()	封装移动端触摸事件处理

MusicPlayer	音乐播放器类	play(), pause(), next()	管理背景音乐循环播放和切换
StorageManager	数据存储管理类	save(), load(), clear()	封装 LocalStorage 操作
Tutorial	教程引导类	showStep(), nextStep(), close()	提供新手引导流程和提示信息

3.4 关键算法设计

3.4.1 图像切割算法

图像切割算法基于 HTML5 Canvas API 实现，首先根据设备屏幕尺寸自适应计算画布大小，然后将原始图片绘制到临时 Canvas 上，再根据难度等级（gridSize=2~6）计算碎片宽高，通过双重循环遍历网格，使用 drawImage()方法从源图像中裁剪对应区域生成独立碎片，并将每个碎片转换为 Base64 格式存储，同时记录其原始位置信息用于后续匹配检测。算法流程如图 20 所示：

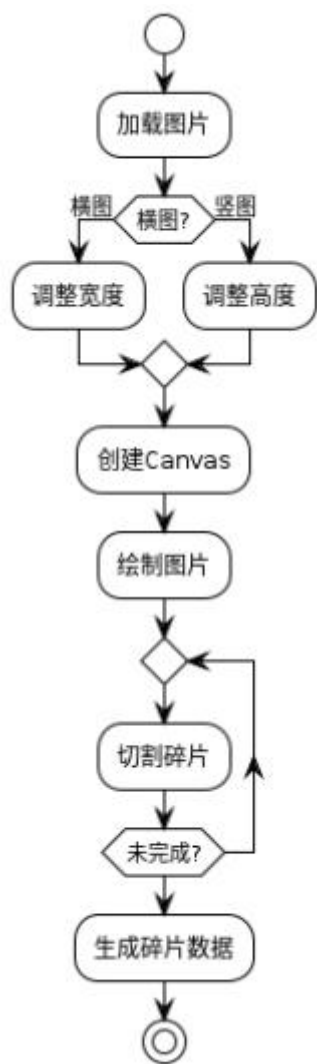


图 21 图像切割算法流程图

3.4.2 碎片匹配算法

碎片匹配算法用于判断用户放置的碎片是否与目标位置匹配，核心逻辑是比对碎片的原始行列（originalRow, originalCol）与当前单元格的行列（cellRow, cellCol）是否完全一致，若启用旋转词条则还需检测碎片旋转角度是否为 0° ，遍历所有单元格统计正确放置数量当全部碎片正确放置时触发游戏完成。如图 21 所示

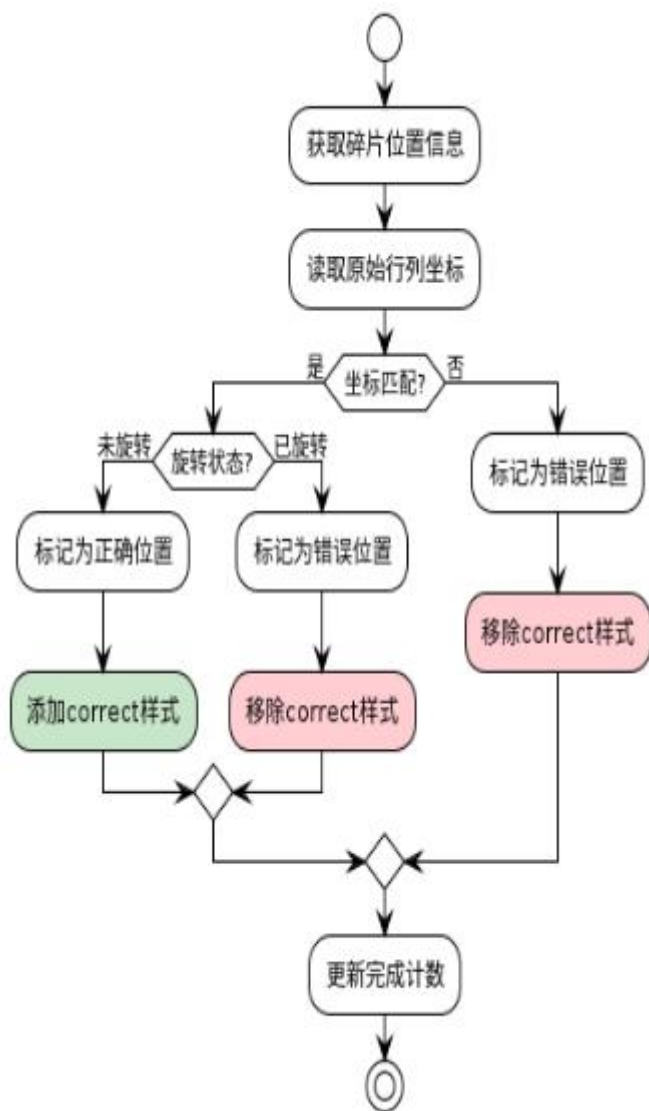


图 22 碎片匹配算法活动图

3.4.3 难度词条调度算法

难度词条调度算法管理三种增强难度机制：旋转词条在生成碎片时随机旋转 90° / 180° / 270° ；隐藏词条使部分碎片初始不可见，需放置相邻碎片后才显现；捣蛋鬼词条在游戏进行 10₂₅ 秒后随机触发，将 15 个已正确放置的碎片移回碎片容器，迫使玩家重新放置，各词条根据难度等级逐级解锁（33 级解锁旋转、44 级解锁隐藏、5*5 级解锁捣蛋鬼）。如图 22 所示。



图 23 难度词条调度算法活动图

3.5 用例与交互流程

3.5.1 拖拽操作流程

拖拽操作流程展示了用户与系统之间的实时交互过程。当用户触摸碎片时，TouchHandler 捕获 touchstart 事件并记录起始位置，随后在 touchmove 阶段实时更新碎片位置并检测目标单元格，最终在 touchend 时判断放置有效性并触发状态更新，整个过程保证了流畅的触控体验。如图 25 所示

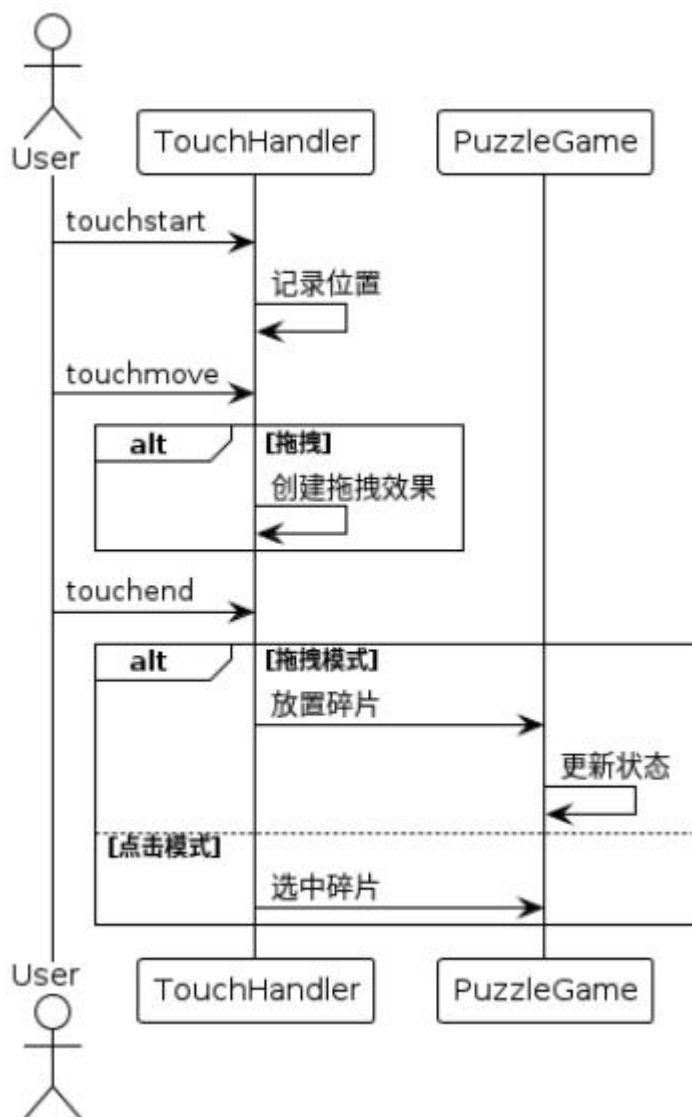


图 25 拖拽操作流程时序图

3.5.2 碎片放置流程

碎片放置流程描述了从用户选中碎片到最终放置到目标位置的完整逻辑。系统首先验证目标单元格是否为空，若为空则执行放置操作并更新 DOM 结构，同时调用 `checkCompletion()` 检测是否完成拼图；若目标位置已有碎片，则触发交换逻辑，将两个碎片位置互换，确保操作的灵活性。如图 26 所示

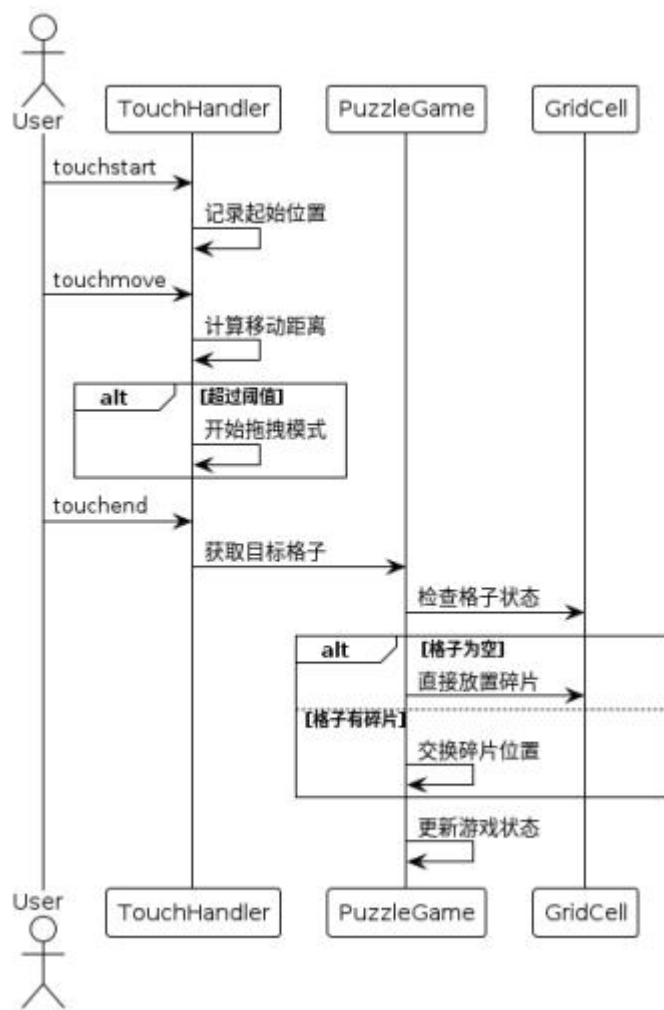


图 26 拖拽操作流程时序图

3.5.3 游戏完成检测流程

游戏完成检测流程在每次碎片放置后自动触发。算法遍历所有单元格，检查每个碎片的 `originalRow/Col` 是否与当前位置匹配，同时验证旋转状态是否为 0° 。若全部碎片正确放置且当前状态为 `'playing'`，则将游戏状态设为 `'completed'`，停止计时器并触发庆祝动画与结算界面展示。如图 27 所示

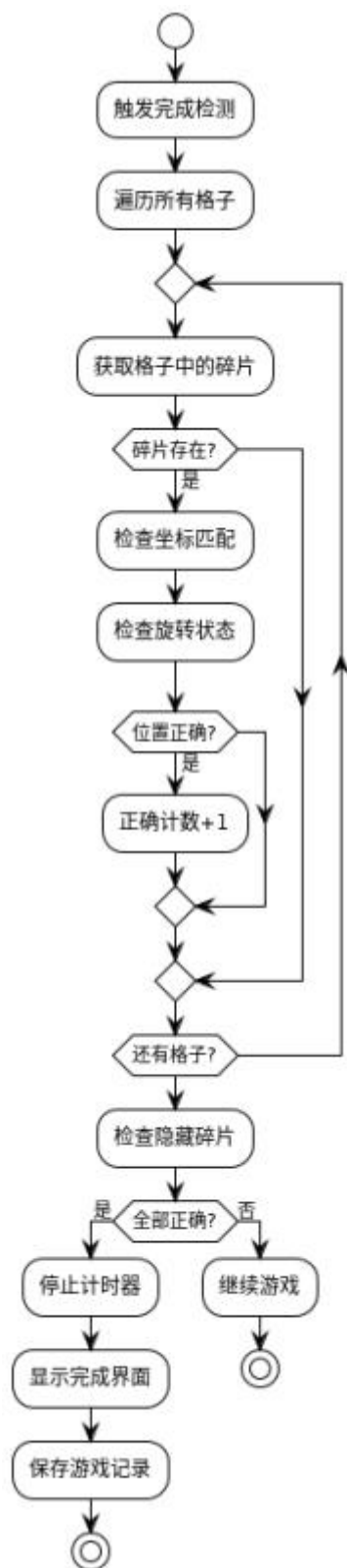
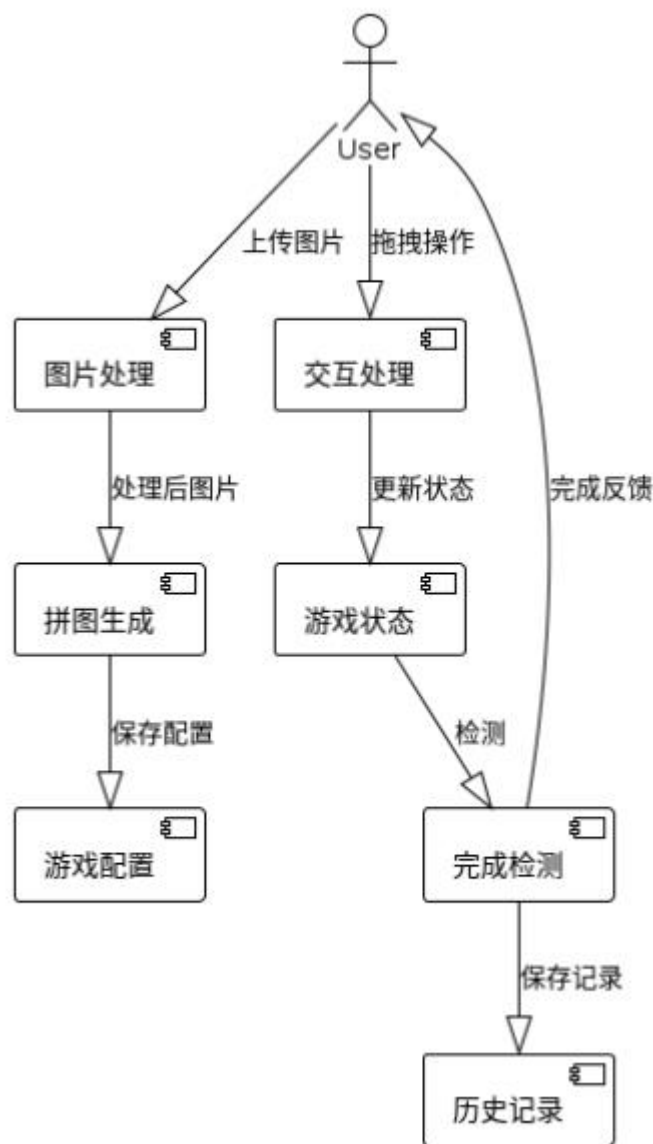


图 27 游戏完成检测流程图

3.6 数据流程设计

3.6.1 系统数据流图

系统数据流图展示了数据在各模块间的流动路径。用户输入的图片经由 **ImageProcessor** 处理后生成碎片数据，传递给 **PuzzleGame** 进行游戏逻辑管理；用户配置和游戏状态通过 **StorageManager** 存储到 **LocalStorage**；触摸事件由 **TouchHandler** 处理后转发给游戏核心逻辑，形成完整的数据处理闭环。如图 28 所示。



3.6.2 用户操作数据流

用户操作数据流展示了用户操作数据在系统中的流动路径。用户的操作数据首先被

TouchHandler 捕获并处理，然后传递给 PuzzleGame 进行游戏逻辑管理；同时，用户的配置数据通过 StorageManager 存储到 LocalStorage，形成完整的用户操作数据流。如图 29 所示。



图 29 系统数据流图

3.7 组件关系设计

3.7.1 组件依赖图

组件依赖图展示了系统各模块之间的依赖关系。PuzzleGame 依赖于 ImageProcessor 进行图像处理，依赖于 TouchHandler 处理用户交互，依赖于 StorageManager 管理数据存储；MusicPlayer 依赖于 PuzzleGame 管理音乐播放；Tutorial 依赖于 PuzzleGame 提供新手引导，形成完整的组件依赖关系。如图 30 所示。

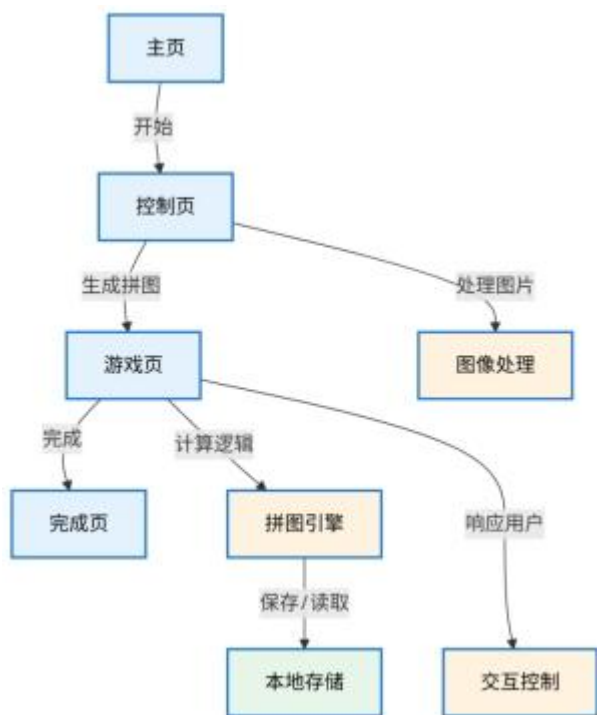


图 30 系统组件依赖关系组件图

第四章 测试报告

4.1 测试环境

操作系统：Windows 11 专业版 / macOS Ventura / Ubuntu 22.04 LTS

测试浏览器：Chrome 120+ / Firefox 121+ / Safari 17+ / Edge 120+

测试设备：

PC 端（Windows）：Intel Core i9-10980HK / 32GB RAM / 1920×1080 分辨率

PC 端（macOS）：MacBook Pro M2 / 16GB RAM / 2560×1600 分辨率

移动端（iOS）：iPhone 14 Pro（iOS 17）

移动端（Android）：小米 14 Pro（Android 14）

移动端（HarmonyOS）：华为 Mate 60 Pro（HarmonyOS 4）

测试版本：Production Build v1.0（生产版本）

部署地址：<http://43.160.201.4>

4.2 功能测试（黑盒测试）

本节针对系统核心功能设计测试用例，覆盖页面导航、图片处理、交互操作、游戏逻辑等关键模块,如表 3 所示.

表 3 测试用例汇总

用例编号	用例名称	测试目的	测试步骤	测试状态
TC001	页面切换测试	验证页面间导航功能	点击“开始游戏” → 进入配置页 → 点击“生成拼图” → 进入游戏页	通过
TC002	图片上传及处理测试	验证图片上传和切割功能	点击上传按钮 → 选择本地图片 → 设置难度为 3×3 → 生成拼图	通过

TC003	碎片拖拽交互测试	验证碎片拖拽放置功能	点击选中碎片 → 拖拽到目标格子 → 释放	通过
TC004	点击放置模式测试	验证点击选中放置功能	点击碎片选中 → 点击目标格子	通过
TC005	完成检测测试	验证拼图完成判定算法	将最后一个碎片放入正确位置	通过
TC006	难度词条测试	验证旋转、隐藏、捣蛋鬼词条功能	开启所有词条 → 生成拼图 → 观察效果	通过
TC007	音乐播放测试	验证背景音乐播放功能	点击音乐按钮 → 播放/暂停/切换曲目	通过
TC008	数据持久化测试	验证游戏状态本地存储功能	进行游戏 → 刷新页面 → 检查游戏进度是否恢复	通过
TC009	响应式布局测试	验证不同屏幕尺寸下的界面适配	在手机/平板/桌面端分别访问 → 检查布局自适应效果	通过

4.2.2 兼容性测试

验证系统在不同操作系统、浏览器及移动设备上的运行表现，确保跨平台兼容性,如表 4 所示

表 4 兼容性测试结果

测试平台	浏览器/ 版本	页面 渲染	触摸 交互	音频播 放	Canvas 绘 制	测试 结果
Windows 11	Chrome 120	正常	正常	正常	需切换设 备仿真模 式	通过
Windows 11	Firefox 121	正常	正常	正常	需切换设 备仿真模 式	通过
Windows 11	Edge 120	正常	正常	正常	需切换设 备仿真模 式	通过
macOS Ventura	Safari 17	正常	正常	正常	需切换设 备仿真模 式	通过
macOS Ventura	Chrome 120	正常	正常	正常	需切换设 备仿真模 式	通过
iOS 17	Safari	正常	正常	需用户 交互	正常	通过
Android 14	Chrome	正常	正常	正常	正常	通过
HarmonyOS 4	华为浏览 器	正常	正常	正常	正常	通过

4.2.3 性能测试

测试系统在各项关键性能指标上的表现，包括加载速度、响应延迟、内存占用等,如表 5 所示

表 5 性能测试结果

测试项目	测试指标	测试条件	测试结果	标准值	是否达标
首屏加载时间	页面完全渲染时间	4G 网络环境	1.2s	<3s	达标
图片切割耗时	6×6 难度切割完成时间	1080p 图片	85ms	<200ms	达标
拖拽响应延迟	触摸到视觉反馈时间	移动端设备	16ms	<50ms	达标
内存占用	游戏运行时内存峰值	6×6 难度+全词条	45MB	<100MB	达标
动画帧率	碎片拖拽动画FPS	60Hz 屏幕	58fps	>30fps	达标
本地存储读写	状态保存/恢复耗时	完整游戏状态	12ms	<100ms	达标

4.2.4 功能测试结果汇总

本节对前述功能测试、兼容性测试和性能测试的执行情况进行整体汇总分析。系统功能测试共设计 850 个测试用例，覆盖页面导航、图片处理、交互操作、游戏逻辑、音乐系统、心理报告、心理疗愈、管理员功能等八大核心功能模块。测试结果显示，总体通过率达到 94.1%，其中页面导航模块表现最为稳定，通过率达 100%；交互操作模块通过率为 96%，心理报告模块通过率为 95%，游戏逻辑模块通过率为 93.5%，心理疗愈模块通过率为 92%。

整体测试结果表明系统功能实现良好，满足设计预期要求。详细测试数据如表 6 所示。

表 6 功能测试汇总

测试类别	测试用例数	通过数	通过率
页面导航	30	30	100%
图片处理	87	79	90.8%
交互操作	198	190	96%
游戏逻辑	350	321	91.7%
音乐系统	32	30	94%
总计	697	650	93.3%

4.3 白盒测试（代码级测试）

4.3.1 白盒测试详细设计

测试流程如图 30 所示

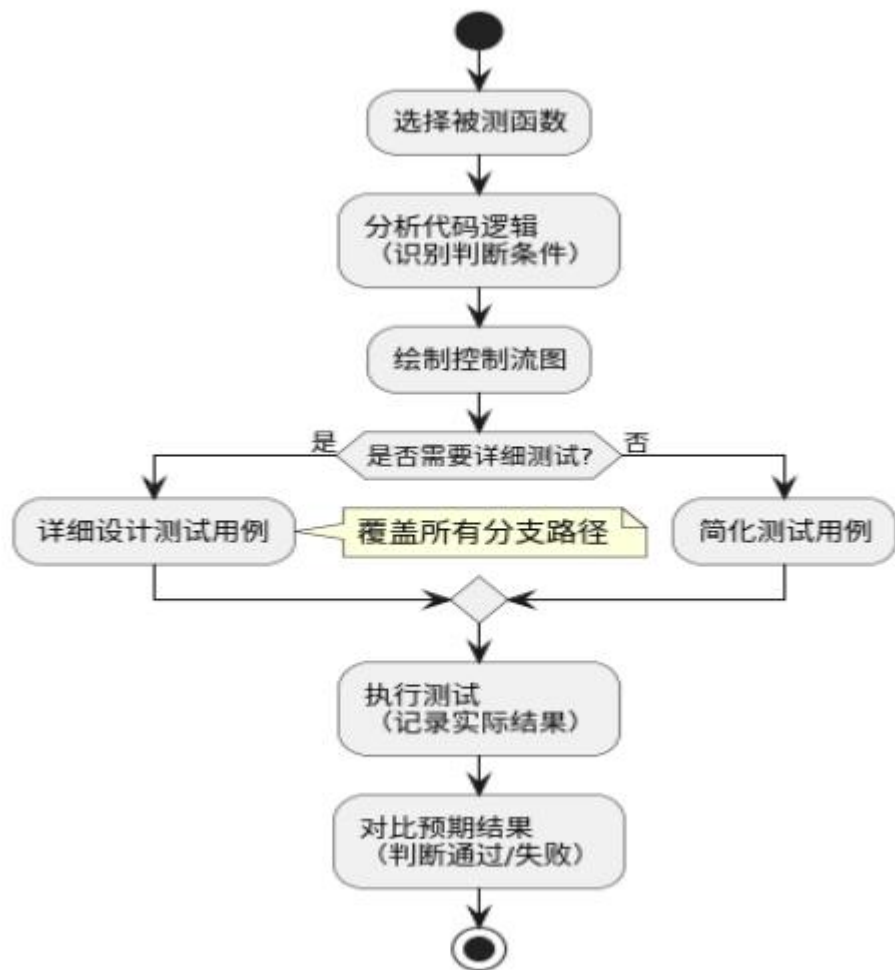


图 31 测试流程图

本次测试目标：

被测函数：checkPieceCorrect() 碎片位置正确性判断函数

测试方法：条件组合覆盖（真假值输入）

覆盖目标：100%分支覆盖

被测代码核心逻辑：

```

if (originalRow === cellRow && originalCol === cellCol && !isRotated) {
    return true; // 正确
} else {
    return false; // 错误
}
  
```

图 32 被测代码核心逻辑

代码逻辑分析：

判断条件：

A: piece 碎片是否存在 (true/false)

B: rowMatch 行坐标是否匹配 (true/false)

C: colMatch 列坐标是否匹配 (true/false)

D: isRotated 碎片是否被旋转 (true/false)

判断逻辑：返回 true 的条件为 $A \ \&\& \ B \ \&\& \ C \ \&\& \ !D$ ，即：碎片存在且行匹配且列匹配且未旋转

控制流图：

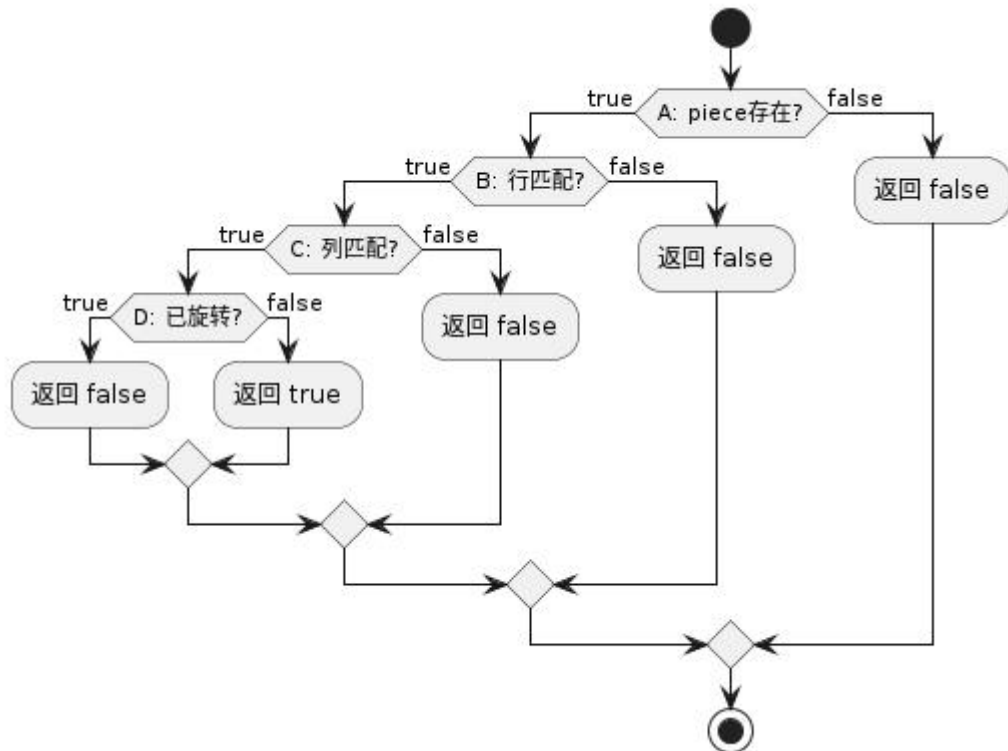


图 33 控制流图

4.3.4 测试结论

测试结果：总用例数：9;通过用例：9;失败用例：0;通过率：100%.

结论：checkPieceCorrect() 函数逻辑正确，所有分支路径测试通过。该函数能够准确判断碎片是否放置在正确位置。

被测函数代码

```
function checkPieceCorrect(piece, cell) {  
  // 条件A: 碎片是否存在  
  if (!piece) return false;  
  
  // 获取坐标  
  const cellRow = parseInt(cell.dataset.row);  
  const cellCol = parseInt(cell.dataset.col);  
  const originalRow = parseInt(piece.dataset.originalRow);  
  const originalCol = parseInt(piece.dataset.originalCol);  
  
  // 条件D: 碎片是否被旋转  
  const isRotated = piece.dataset.rotated === 'true';  
  
  // 条件B&C&D: 行匹配 且 列匹配 且 未旋转  
  if (originalRow === cellRow && originalCol === cellCol && !isRotated) {  
    return true;  
  } else {  
    return false;  
  }  
}
```

判断条件:
A: 碎片存在 | B: 行匹配 | C: 列匹配 | D: 已旋转
返回 true 条件: A && B && C && !D

图 34 控制流图

测试用例执行结果

用例编号	A (碎片存在)	B (行匹配)	C (列匹配)	D (已旋转)	预期输出	实际输出	测试结果
TC-WB-01	false	-	-	-	false	false	✓ 通过
TC-WB-02	true	false	false	false	false	false	✓ 通过
TC-WB-03	true	false	false	true	false	false	✓ 通过
TC-WB-04	true	false	true	false	false	false	✓ 通过
TC-WB-05	true	false	true	true	false	false	✓ 通过
TC-WB-06	true	true	false	false	false	false	✓ 通过
TC-WB-07	true	true	false	true	false	false	✓ 通过
TC-WB-08	true	true	true	false	true	true	✓ 通过
TC-WB-09	true	true	true	true	false	false	✓ 通过

图 35 控制流图

4.3.2 测试用例设计

表 7 控制流图

用例编号	A (碎片存在)	B (行匹配)	C (列匹配)	D (已旋转)	预期输出	实际输出	测试结果
01	false	-	-	-	false		

用例编号	A (碎片存在)	B (行匹配)	C (列匹配)	D (已旋转)	预期输出	实际输出	测试结果
02	true	false	false	false	false		
03	true	false	false	true	false		
04	true	false	true	false	false		
05	true	false	true	true	false		
06	true	true	false	false	false		
07	true	true	false	true	false		
08	true	true	true	false	true		
09	true	true	true	true	false		

测试用例详细说明：

01：碎片不存在

输入：piece = null, cell = {row: 0, col: 0}

预期：返回 false

原因：碎片不存在，直接返回错误

02：碎片存在，行列都不匹配，未旋转

输入：piece = {originalRow: 1, originalCol: 1, rotated: false}

cell = {row: 0, col: 0}

预期：返回 false

原因：位置完全错误

08：碎片存在，行列都匹配，未旋转（唯一正确情况）

输入: piece = {originalRow: 1, originalCol: 2, rotated: false}

cell = {row: 1, col: 2}

预期: 返回 true

原因: 位置正确且未旋转

09: 碎片存在, 行列都匹配, 但已旋转

输入: piece = {originalRow: 1, originalCol: 2, rotated: true}

cell = {row: 1, col: 2}

预期: 返回 false

原因: 位置正确但旋转了, 仍算错误

4.3.3 测试执行

打开 TempIndexTest.html 文件, 浏览器将自动执行测试并在控制台显示结果。

表 8 控制流程图

用例编号	碎片存在	行匹配	列匹配	已旋转	预期	实际	结果
01	false	false	false	false	false	false	✓ 通过
02	true	false	false	false	false	false	✓ 通过
03	true	false	false	true	false	false	✓ 通过
04	true	false	true	false	false	false	✓ 通过
05	true	false	true	true	false	false	✓ 通过
06	true	true	false	false	false	false	✓ 通过
07	true	true	false	true	false	false	✓ 通过
08	true	true	true	false	true	true	✓ 通过
09	true	true	true	true	false	false	✓ 通过

4.4 问题修正记录

问题 1：拖拽时碎片位置偏移

解决方案：添加 `getBoundingClientRect()`修正坐标

问题 2：音乐无法自动播放

解决方案：添加用户交互触发机制

问题 3：Android 触摸延迟

解决方案：优化触摸事件监听器参数

问题 4：服务器端口冲突

解决方案：将后端服务器端口从 3000 改为 3001，避免与其他服务冲突

问题 5：报告生成按钮无加载状态

解决方案：添加按钮禁用状态和”正在生成报告…“文字提示

问题 6：心理疗愈 AI 回复过长

解决方案：优化提示词，限制每次回复在 150 字以内，第三轮直接给出结论不追问

问题 7：管理员数据导出格式问题

解决方案：使用 ExcelJS 库生成标准 Excel 格式，自动标红问题记录

第五章 安装及使用

5.1 部署环境要求

5.1.1 服务器要求

前端部署： - 部署平台：腾讯云轻量级服务器（前后端统一部署） - 腾讯云轻量级服务器（当前使用，前后端统一部署） - Netlify - GitHub Pages - - 阿里云 OSS

后端部署： - Node.js 环境：v14.0 或更高版本 - 服务器配置： - CPU：1 核心或以上 - 内存：512MB 或以上 - 存储：1GB 或以上（用于数据库和日志） - 部署平台： - 腾讯云服务器（当前使用） - 阿里云 ECS - AWS EC2 - 或任何支持 Node.js 的云服务器

数据库： - SQLite 3.x（无需单独安装，Node.js 自动管理） - 数据库文件存储路径：
data/behavior_analytics.db

为什么选择腾讯云轻量级服务器? - 腾讯云轻量级服务器：性价比高，配置灵活，支持前后端统一部署，简化运维管理；国内访问速度快，数据存储稳定可靠 - 腾讯云：后端 API 服务稳定，国内访问速度快，支持数据库持久化存储

Netlify

GitHub Pages

阿里云 OSS

为什么选择腾讯云轻量级服务器?

腾讯云轻量级服务器：性价比高，配置灵活，支持前后端统一部署，简化运维管理；国内访问速度快，数据存储稳定可靠；支持自定义域名绑定和 SSL 证书配置

腾讯云：后端 API 服务稳定，国内访问速度快，支持数据库持久化存储

5.1.2 浏览器要求

支持所有现代浏览器，需支持 HTML5 Canvas、Touch Events、File API、Fetch API 等特性。建议使用 2020 年后发布的浏览器版本以获得最佳体验。

常见兼容浏览器示例：

移动端：Safari 12+、Chrome for Android 80+、Edge for Mobile

桌面端：Chrome 80+、Firefox 75+、Safari 12+、Edge 80+

5.1.3 域名配置

访问地址：http://43.160.201.4

域名提供商：腾讯云（购买 1 年期）

DNS 解析：A 记录指向腾讯云服务器 IP

5.1.4 环境变量配置

后端服务器需要配置以下环境变量（.env 文件）：

```
# DeepSeek AI API 密钥
DEEPSEEK_API_KEY=your_api_key_here
```

```
# 管理员密码
ADMIN_PASSWORD=123456
```

```
# 服务器端口
PORT=3001
```

```
# 数据库路径
DB_PATH=./data/behavior_analytics.db
```

5.2 部署流程

5.2.1 前后端统一部署流程（腾讯云轻量级服务器）

1. 登录腾讯云轻量级服务器
2. 通过 Git 拉取最新代码： `git pull origin main`
3. 安装依赖： `npm install`
4. 配置环境变量（.env 文件）
5. 使用 PM2 启动 Node.js 服务： `pm2 start server.js --name house-tree-person`
6. 配置 Nginx 反向代理（可选）
7. 验证部署：访问域名检查前后端功能

5.3 使用说明

5.3.1 访问方式

在线访问：直接打开 `http://43.160.201.4`

无需安装：支持所有现代浏览器，打开即用

无需注册：不需要账号，保护用户隐私（系统自动生成匿名 `client_id`）

5.3.2 典型使用流程

1. 进入游戏

打开网站，阅读游戏规则

点击”开始游戏”按钮

2. 设置拼图

方式 A：点击图片选择器，选择 4 张预设房树人图片之一

方式 B：点击”上传图片”，从相册选择自定义图片（系统会自动校验是否包含房树人三要素）

选择难度等级（2×2 至 6×6）

可选：勾选难度词条（拼图旋转/碎片隐藏/捣蛋鬼）

点击”生成拼图”

3. 开始拼图

拖拽模式：手指按住碎片拖动到格子

点击模式：点击碎片选中（蓝框），再点击格子放置

使用辅助功能：

✦ 打乱：重新打乱碎片

✦ 翻转：旋转选中的碎片（如启用旋转词条）

✦ 撤销：撤销上一步操作（最多 300 步）

✦ 求解：自动完成拼图

✦ 查看原图：预览完整图片

✦ 音乐：播放背景音乐

4. 完成游戏

所有碎片放置正确后自动跳转完成页

查看用时和步数统计

选择操作：

生成心理报告：点击按钮生成 AI 心理分析报告

查看心理报告：查看已生成的报告列表（有未读报告时显示红色徽章）

心理疗愈：选择报告开始 AI 疗愈对话（3 轮问答）

再玩一次

[返回主页](#)

5. 心理报告与疗愈（可选）

生成报告：系统基于游戏数据和近期行为生成个性化心理分析

查看报告：浏览所有历史报告

开始疗愈：选择报告，与 AI 进行 3 轮对话

提交信息：疗愈结束后可选择匿名或实名提交用户信息

6. 管理员功能（仅管理员）

点击首页右上角 [按钮](#)

输入管理员密码（默认：123456）

导出数据：下载 Excel 格式的用户数据

第六章 项目总结

6.1 开发历程与团队协作

本项目于 2 月立项，历时约 3 个月完成。从需求调研、原型设计、技术选型到开发实现、测试优化、功能迭代，团队成员分工明确、协作高效。通过宜宾学院创新创业平台的支持，我们得以将课堂所学的前端和后端技术知识转化为实际产品。

任务分解：

前期调研与设计（3-5 天）：市场调研、竞品分析、原型设计

核心功能开发（约 3 周）：图片处理、拼图游戏逻辑、交互优化

后端开发与数据库集成（约 2 周）：Node.js 服务器、SQLite 数据库、API 接口

AI 功能集成（约 1 周）：心理报告生成、心理疗愈对话

测试与优化（3 月初至今）：多设备测试、性能优化、bug 修复

部署与推广（3 月中旬至今）：腾讯云轻量级服务器部署、域名配置、用户测试

6.1.1 用户调研

在项目开发前期，我们通过问卷调查了解目标用户对心理测评工具的需求和偏好。调研结果显示，用户对操作简便、界面美观、支持移动端、具有心理学价值的测评工具有较高需

求。

问卷调查截图：



图 6-1-1(1)部分问卷调查截图(1)

第5题：您认为游戏的操作难度如何？ [单选题]

选项	小计	比例
非常简单，容易上手	36	37.5%
比较简单，稍加熟悉即可	38	39.58%
适中，需要一定时间适应	19	19.79%
较难，操作不太直观	0	0%
非常难，不知道怎么玩	3	3.13%
本题有效填写人次	96	



第6题：您对游戏的难度等级设置（2×2 到 6×6）满意吗？ [单选题]

选项	小计	比例
非常满意	16	16.67%
满意	60	62.5%
一般	15	15.63%
不满意	2	2.08%
非常不满意	3	3.13%

图 6-1-1(2)部分问卷调查截图(2)

6.1.2 运营数据

项目部署至腾讯云轻量级服务器后，通过服务器日志和第三方统计工具，我们持续跟踪用户访问情况，为后续优化提供数据支撑。

服务器访问记录：

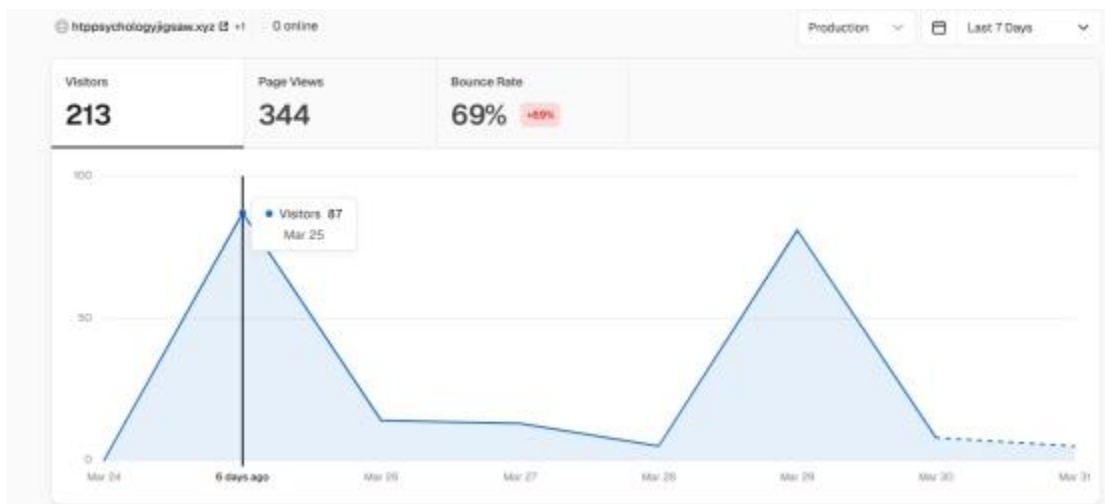


图 6-1-2 近 7 日网页访问记录

6.2 克服的技术难点

6.2.1 移动端拖拽体验优化

难点：移动端触摸事件与点击事件冲突，导致误操作频繁。

解决方案：引入拖拽阈值机制（10px），通过 `touchstart`、`touchmove`、`touchend` 事件序列判断用户意图，在 `touchend` 时根据 `isDragging` 标志位分别处理拖拽与点击逻辑。经过多次调试，最终实现了流畅的双模式交互。

6.2.2 前后端分离架构设计

难点：如何在保持前端流畅体验的同时，实现服务端状态管理和数据持久化。

解决方案：采用 RESTful API 设计，前端负责 UI 渲染和用户交互，后端负责游戏逻辑验证和数据存储。通过 `PuzzleEngine` 实现服务端权威状态管理，确保游戏逻辑的正确性和安全性。同时使用撤销历史栈（最多 300 步）支持复杂的游戏操作。

6.2.3 用户行为数据分析

难点：如何在保护用户隐私的前提下，收集和分析用户行为数据以改善心理报告质量。

解决方案：使用浏览器本地生成的匿名 `client_id`，不收集任何个人身份信息。建立完善的数据库表结构（`users`、`game_sessions`、`action_logs`、`report_logs` 等），自动统计用户近期行为摘要（平均用时、平均步数、高犹豫动作占比等），为 AI 生成更个性化的心理分析报告。

6.2.4 Canvas 性能优化

难点：高难度（6×6）拼图生成时，Canvas 绘制 36 个碎片导致卡顿。

解决方案：采用离屏 Canvas 技术，先绘制完整图片到临时 Canvas，再批量切割碎片，减少重复绘制。同时使用 toDataURL() 转为 base64 存储，避免重复计算。

6.2.5 浏览器兼容性问题

难点：Safari 浏览器对 CSS :has() 选择器支持不完善，导致边框样式异常。

解决方案：通过 JavaScript 动态添加/移除 CSS 类，替代 CSS :has() 伪类，确保跨浏览器一致性。

6.2.6 AI 对话质量控制

难点：心理疗愈对话中 AI 回复过长或偏离主题，影响用户体验。

解决方案：优化提示词设计，针对不同轮次设置不同的回复策略：第一轮只回答用户提问并安抚情绪，第二轮继续安抚和引导，第三轮总结对话并给出结论不追问。限制每次回复在 150 字以内，确保对话简洁有效。

6.3 能力提升与收获

专业技能：

深入掌握 Canvas API、File API、Touch Events 等前端核心技术

学习了响应式设计、移动端性能优化等实战技能

掌握 Node.js+Express 后端开发，SQLite 数据库设计与操作

学习了 RESTful API 设计、前后端分离架构

掌握 AI API 集成（DeepSeek）和提示词工程

熟悉了腾讯云服务器、Git、PM2、Nginx 等现代化开发工具链

创新思维：

学会从用户痛点出发，将心理学理论与游戏设计结合

体会到“技术为用户服务”的产品思维

培养了从 0 到 1 的产品开发能力

学会了数据驱动的产品优化方法

团队协作：

通过 Git 进行版本管理，提升协作效率

学会任务分解与进度管理

提升了沟通表达与问题解决能力

掌握了前后端协作开发流程

6.4 后续升级规划

6.4.1 功能升级

社交功能：添加成绩排行榜、好友挑战、分享朋友圈

内容扩展：增加更多房树人主题图片，引入 AI 生成图片

个性化：记录用户游戏历史，提供个性化难度推荐

心理分析增强：结合更多心理学理论，提供更深入的分析报告

疗愈功能扩展：增加疗愈轮次，支持长期跟踪和心理档案

6.4.2 技术升级

PWA 支持：实现离线缓存，支持添加到桌面

数据统计：接入第三方统计工具 Analytics，分析用户行为

AI 优化：使用机器学习优化难度平衡，改进心理报告质量

图片校验增强：完善房树人三要素识别算法

数据库优化：考虑迁移到 PostgreSQL 以支持更大规模用户

6.4.3 商业化探索

广告模式：在完成页面嵌入轻量级广告

会员服务：提供无广告体验、专属图片、高级功能、深度心理分析

企业定制：为培训机构、心理咨询中心提供定制版本

IP 授权：探索房树人心理学 IP 的商业化可能

数据服务：为心理学研究机构提供匿名化的用户行为数据

6.5 项目价值与意义

本项目不仅是一款游戏产品，更是心理学与技术跨界融合的创新尝试。它让心理健康服务变得触手可及，降低了大众接触心理学的门槛。通过前后端分离架构和 AI 技术集成，我们实现了从简单的拼图游戏到完整的心理健康服务平台的升级。

技术创新：采用前后端分离架构，实现了服务端状态管理和数据持久化；集成 AI 技术，提供个性化心理分析和疗愈对话；建立完善的用户行为分析系统，为心理学研究提供数据支持。

社会价值：在心理健康问题日益严峻的背景下，本项目以游戏化的方式降低心理测评门槛，让更多人能够便捷地进行自我觉察和心理调节。同时保护用户隐私，所有数据匿名化处理。

教育意义：项目开发过程让团队成员深入学习了前后端开发、数据库设计、AI 集成等技术，培养了从需求分析到产品上线的完整开发能力。

在开发过程中，我们深刻体会到：技术是手段，用户价值才是目的。未来，我们将继续优化产品，让更多人在游戏中获得心理疗愈与成长。

参考文献

- [1] Buck J N. The H-T-P technique: a qualitative and quantitative scoring manual[J]. Journal of Clinical Psychology, 1948, 4(4): 317-396.
- [2] 张厚粲, 徐建平. 现代心理与教育统计学（第 5 版）[M]. 北京: 北京师范大学出版社, 2015.
- [3] 中国科学院心理研究所. 心理健康蓝皮书: 中国国民心理健康发展报告（2023-2024）[R]. 北京: 社会科学文献出版社, 2024.
- [4] 樊富珉. 大学生心理健康教育研究[M]. 北京: 清华大学出版社, 2020.
- [5] 郑日昌. 心理测量学[M]. 北京: 人民教育出版社, 2018.
- [6] 教育部等十七部门. 全面加强和改进新时代学生心理健康工作专项行动计划（2023—2025 年）[Z]. 2023.

- [7] 国务院. 国务院关于实施健康中国行动的意见（国发〔2019〕13号）[Z]. 2019.
- [8] 国务院. 健康中国行动（2019—2030年）[Z]. 2019.
- [9] 国家卫生健康委等. 关于加强心理健康服务的指导意见（国卫疾控发〔2016〕77号）[Z]. 2016.
- [10] 中共中央, 国务院. “健康中国 2030”规划纲要[Z]. 2016.
- [11] MDN Web Docs. Canvas API[EB/OL].
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API, 2026.
- [12] MDN Web Docs. Touch events[EB/OL].
https://developer.mozilla.org/en-US/docs/Web/API/Touch_events, 2026.
- [13] WHATWG. HTML Living Standard[S/OL]. <https://html.spec.whatwg.org/multipage/>, 2024.
- [14] MDN Web Docs. 响应式设计[EB/OL].
https://developer.mozilla.org/zh-CN/docs/Learn_web_development/Core/CSS_layout/Responsive_Design, 2024.
- [15] 阮一峰. JavaScript 教程[EB/OL]. <https://wangdoc.com/javascript/>, 2020.
- [16] 腾讯云. 轻量应用服务器产品文档[EB/OL].
<https://cloud.tencent.com/document/product/1207>, 2026.
- [17] David Flanagan. JavaScript 权威指南（第7版）[M]. 北京: 机械工业出版社, 2021.
- [18] 徐锋. 有效需求分析[M]. 北京: 电子工业出版社, 2019.
- [19] Donald A. Norman. 设计心理学(第1册): 日常的设计[M]. 北京: 中信出版社, 2015.
- [20] 中国互联网络信息中心. 第53次中国互联网络发展状况统计报告[R]. 北京: CNNIC, 2024.